



本教學投影片係屬教科書著作之延伸，亦受著作權法之保護。

請依出版社授權使用規範利用本教學資源，非經授權許可不得以影印、拷貝、列印等方法進行重製，亦不得改作、公開展示著作內容，亦請勿對第三人公開傳輸、散布。



Chapter 3

Gate-Level Minimization

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Chapter Objectives

- Know how to derive and simplify a Karnaugh map for the Boolean function of 2, 3, and 4 variables.
- Know how to derive the prime implicants of a Boolean function.
- Know how to obtain the sum of products and the product of sums forms of a Boolean function directly from its Karnaugh map.
- Know how to create the Karnaugh map of a Boolean function from its truth table.
- Know how to use don't care conditions to simplify a Boolean function.
- Know how to form a two-level NAND and a two-level NOR implementation of a Boolean function.

Chapter Objectives

- Know how to declare a Verilog module or a VHDL entity-architecture for a combinational logic circuit.
- For a given logic diagram of a combinational circuit, know how to write a structural model of the circuit using
 - (a) Verilog predefined primitives or
 - (b) user-defined VHDL components.
- Given a test bench, know how to draw the waveform of an input signal to the unit under test.

3.1 Introduction

- Boolean function
 - Sum of minterms (or product of maxterms) $\leftrightarrow$ Canonical form
 - Sum of products (or product of sums) in the simplest form $\leftrightarrow$ Standard form
- *Gate-level minimization* is the design task of finding an optimal gate-level implementation of Boolean functions describing a digital circuit.
- It will be assumed that the simplest algebraic expression is one that has a **minimum number of terms with the smallest possible number of literals in each term.**
- The chapter will also introduce the role and use of hardware description languages in modern logic design methodology.

3.2 The Map Method (1/10)

- The complexity of the digital logic gates is directly related to the complexity of the algebraic expression.
- Logic minimization
 - Algebraic approaches: lack specific rules
 - The Karnaugh map or K-map (卡諾圖)
 - A simple straight forward procedure
 - A pictorial form of a truth table
 - Applicable if the # of variables < 7
 - A diagram made up of squares
 - Each square represents one minterm
 - **The simplified expression may not be unique**

3.2 The Map Method (2/10)

- Two-variable (x, y) K-map
 - four minterms
 - row 0 $\rightarrow x'$; row 1 $\rightarrow x$
 - column 0 $\rightarrow y'$; column 1 $\rightarrow y$
 - a truth table in a square diagram

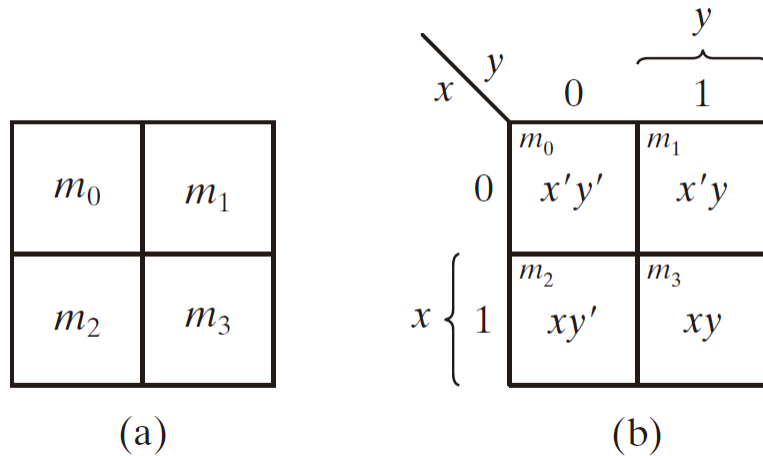


FIGURE 3.1
Two-variable K-map

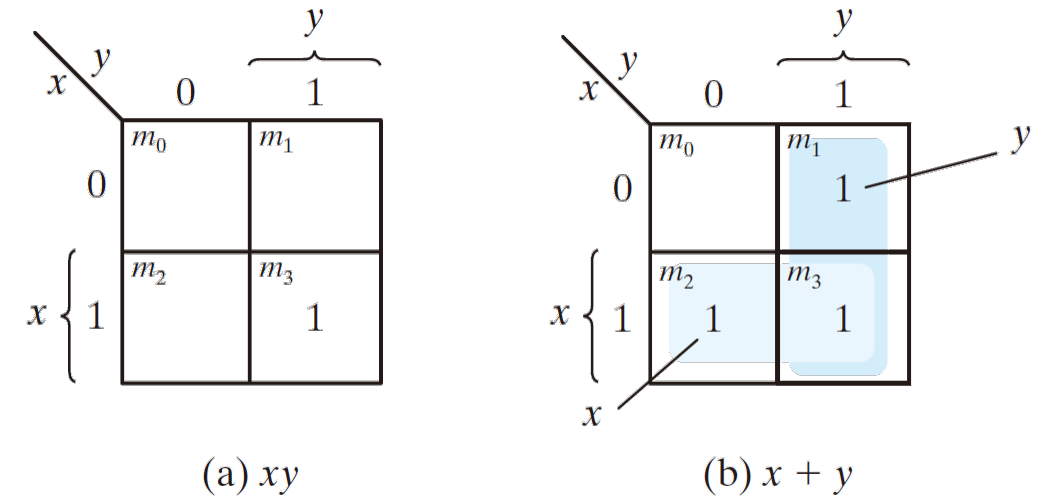


FIGURE 3.2
Representation of functions in the map

3.2 The Map Method (3/10)

- Three-variable (x, y, z) K-map
 - Eight minterms in the sequence
 - Similar to the Gray code, any two adjacent squares in the map differ by only one variable
 - primed in one square and unprimed in the other
 - e.g. m_5 and m_7 can be simplified
 - $m_5 + m_7 = xy'z + xyz = xz(y' + y) = xz$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

		y				
		yz	00	01	11	10
x	0	m_0	m_1	m_3	m_2	
		$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$	
x	1	m_4	m_5	m_7	m_6	
		$xy'z'$	$xy'z$	xyz	xyz'	
		z				

(b)

(b)

FIGURE 3.3
Three-variable K-map

3.2 The Map Method (4/10)

- **Example 3.1** Simplify the Boolean function $F(x, y, z) = \Sigma(2, 3, 4, 5)$

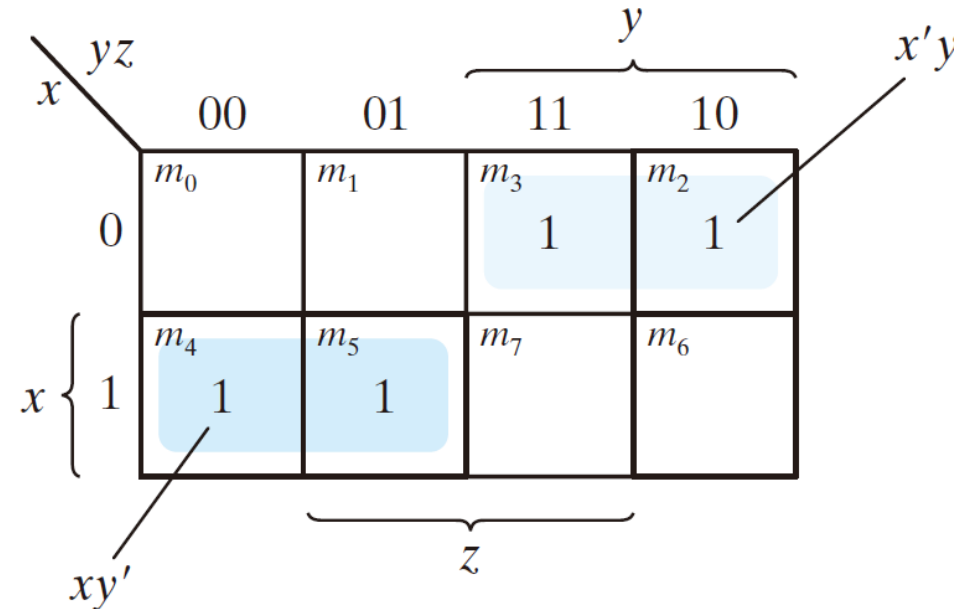


FIGURE 3.4

Map for Example 3.1, $F(x, y, z) = \Sigma(2, 3, 4, 5) = x'y + xy'$

3.2 The Map Method (5/10)

- Two squares in the map are considered to be adjacent even though they do not touch each other.
 - m_0 and m_2 (m_4 and m_6) are adjacent
 - $m_0 + m_2 = x'y'z' + x'yz' = x'z'(y' + y) = x'z'$
 - $m_4 + m_6 = xy'z' + xyz' = xz'(y' + y) = xz'$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

		y			
		z			
x	yz	00	01	11	10
	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
1	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

(b)

FIGURE 3.3
Three-variable K-map

3.2 The Map Method (6/10)

- **Example 3.2** Simplify the Boolean function $F(x, y, z) = \Sigma(3, 4, 6, 7)$

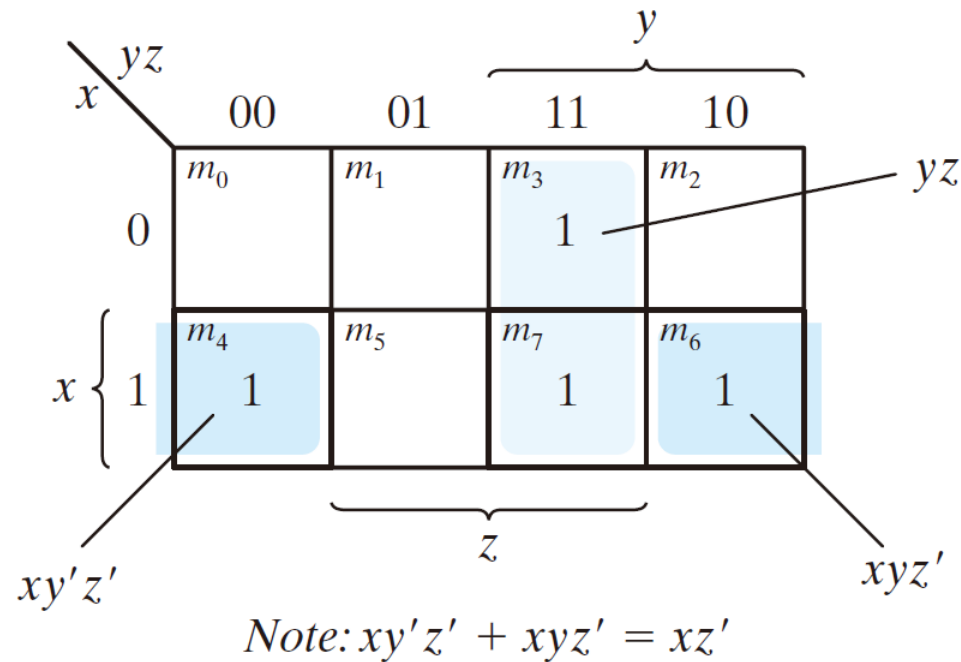


FIGURE 3.5

Map for Example 3.2, $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz'$

3.2 The Map Method (7/10)

- Four adjacent squares

- Any such combination results in an expression with only one literal.
- $$m_0 + m_2 + m_4 + m_6 = x'y'z' + x'yz' + xy'z' + xyz'$$

$$= x'z'(y' + y) + xz'(y' + y) = x'z' + xz' = z'$$
- $$m_1 + m_3 + m_5 + m_7 = x'y'z + x'yz + xy'z + xyz = x'z(y' + y) + xz(y' + y)$$

$$= x'z + xz = z$$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

(a)

		y					
		yz		00	01	11	10
x	0	m_0	m_1	m_3	m_2		
		$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$		
x	1	m_4	m_5	m_7	m_6		
		$xy'z'$	$xy'z$	xyz	xyz'		
		z					

(b)

FIGURE 3.3

Three-variable K-map

3.2 The Map Method (8/10)

- **Example 3.3** Simplify the Boolean function $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$

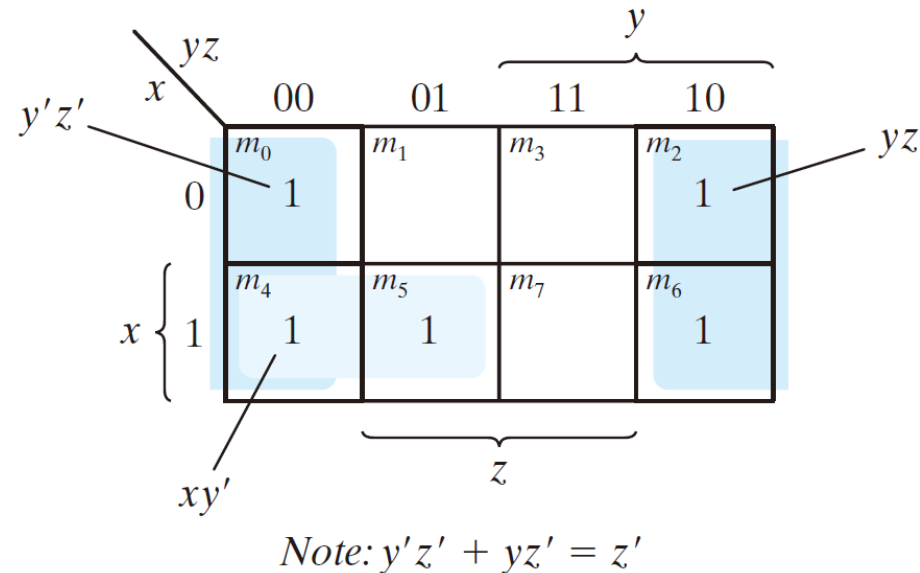


FIGURE 3.6

Map for Example 3.3, $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

3.2 The Map Method (9/10)

- **Example 3.4** For the Boolean function $F = A'C + A'B + AB'C + BC$
 - Express it in sum of minterms
 - Find the minimal sum of products expression

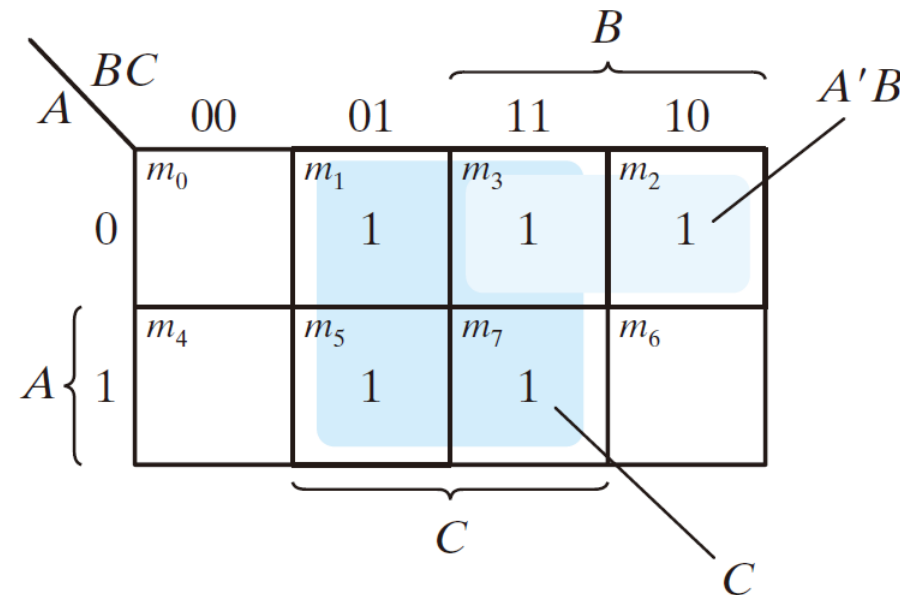


FIGURE 3.7

Map of Example 3.4, $A'C + A'B + AB'C + BC = C + A'B$

3.2 The Map Method (10/10)

- **Practice Exercise 3.1** Simplify the Boolean function $F(x, y, z) = \Sigma(0, 1, 6, 7)$.
Answer: $F(x, y, z) = xy + x'y'$
- **Practice Exercise 3.2** Simplify the Boolean function $F(x, y, z) = \Sigma(0, 1, 2, 5)$.
Answer: $F(x, y, z) = x'z' + y'z$
- **Practice Exercise 3.3** Simplify $F(x, y, z) = \Sigma(0, 2, 3, 4, 6)$.
Answer: $F(x, y, z) = z' + x'y$
- **Practice Exercise 3.4** For the Boolean function $F(x, y, z) = xy'z + x'y + x'z + yz$, (a) express this function as a sum of minterms, and (b) find the minimal sum-of-products expression.
- **Answer:** $F(x, y, z) = m_1 + m_2 + m_3 + m_5 + m_7 = z' + x'y$

3.3 Four-Variable (w, x, y, z) K-Map (1/7)

- 16 minterms
- combinations of 2, 4, 8, and 16 adjacent squares

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

(a)

		y				
		yz	00	01	11	10
w	x	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
		01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	x	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
		10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z				

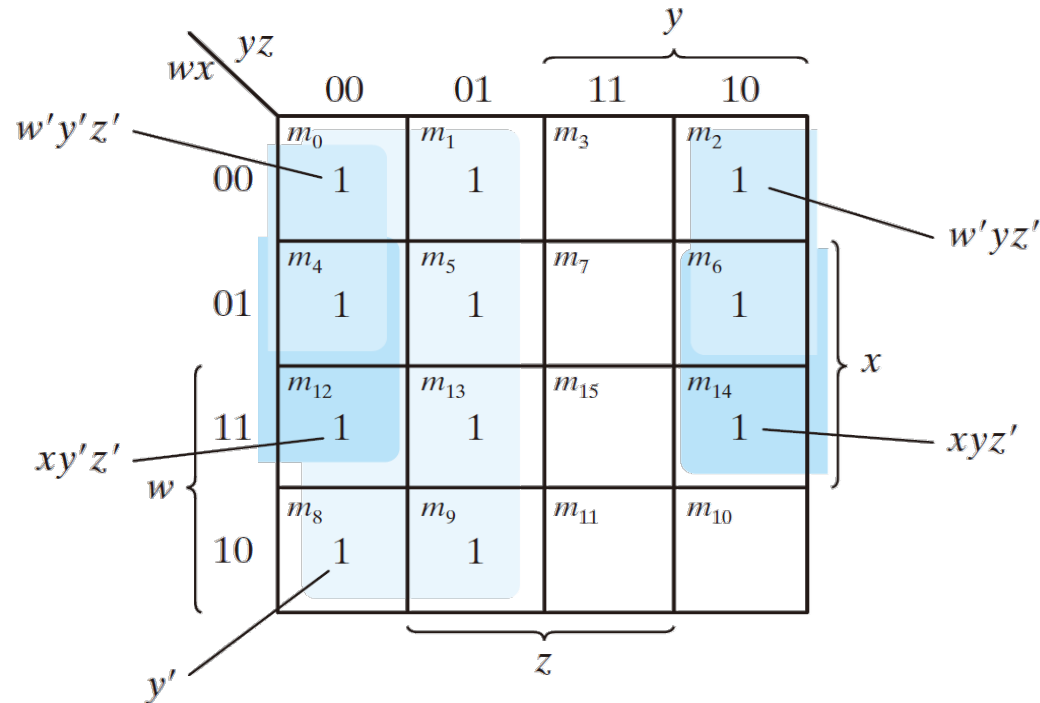
(b)

(b)

FIGURE 3.8
Four-variable map

3.3 Four-Variable K-Map (2/7)

- **Example 3.5** Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$



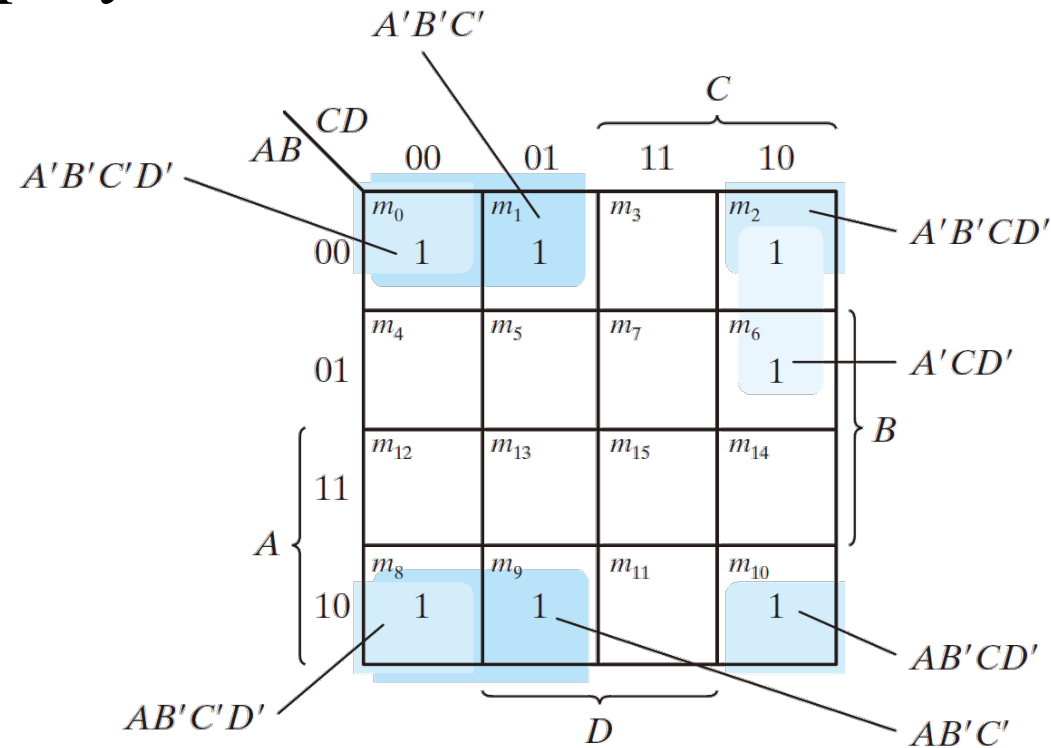
Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

FIGURE 3.9

Map for Example 3.5, $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$

3.3 Four-Variable K-Map (3/7)

- **Example 3.6** Simplify the Boolean function $F = A'B'C' + B'CD' + A'BCD' + AB'C'$



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

FIGURE 3.10

Map for Example 3.6, $A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$

3.3 Four-Variable K-Map (4/7)

- **Practice Exercise 3.5** Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 1, 3, 8, 9, 10, 11, 12, 13, 14, 15)$
Answer: $F(w, x, y, z) = x'y' + x'z + w$
- **Practice Exercise 3.6** Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 2, 4, 6, 8, 10, 11)$
Answer: $F(w, x, y, z) = w'z' + x'z' + wx'y$
- While choosing adjacent squares in a map, we must ensure
 - all the minterms of the function are covered
 - the number of terms in the expression is minimized
 - no redundant terms

3.3 Four-Variable K-Map (5/7)

- Implicant (隱含項)
 - A product term in a SOP expression of a Boolean function
 - 在做卡諾圖化簡時的每個圈，不一定要最簡。
- Prime Implicant (質隱項, PI)
 - A product term obtained by combining the maximum possible number of adjacent squares in the map.
 - 卡諾圖可圈到最大包的implicant。因圈法不同，結果非唯一，但都是最簡的。
- Essential PI (必要質隱項或基本質隱項, EPI)
 - A minterm (或稱為格子) covered by only one PI, that PI is regarded as an EPI
 - The EPI must be included
 - 即為卡諾圖化簡時沒有被重疊到的圈圈(或PI)。它是必須的，因為沒它，圈圈內的某些格子就沒被圈圈包了

3.3 Four-Variable K-Map (6/7)

- Consider $F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$
 - Find the EPI first (if there is one).
 - The simplified expression may not be unique
 - $F = BD + B'D' + CD + AD = BD + B'D' + CD + AB'$
 $= BD + B'D' + B'C + AD = BD + B'D' + B'C + AB'$

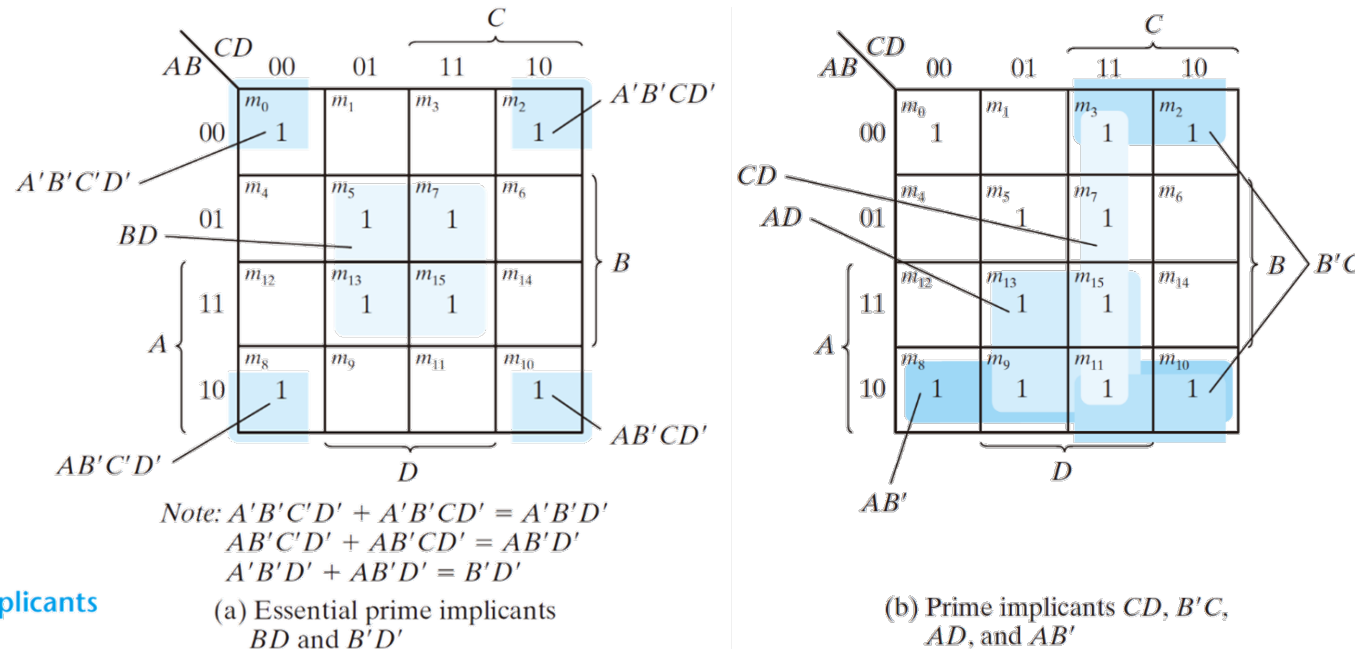


FIGURE 3.11
Simplification using prime implicants

3.3 Four-Variable K-Map (7/7)

- **Practice Exercise 3.7** Find all the possible PIs of the Boolean function
 $F(w, x, y, z) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 14, 15)$
Answer: $x'y', xz, xy, w'x, w'z', yz'$

3.3.1 Five-Variable Map (1/3)

- Map for more than four variables becomes complicated
 - five-variable map: two four-variable map (one on the top of the other)

$A = 0$

		DE		D	
		00	01	11	10
BC					
00		0	1	3	2
01		4	5	7	6
11	B	12	13	15	14
10		8	9	11	10

E

C

$A = 1$

		DE		D	
		00	01	11	10
BC					
00		16	17	19	18
01		20	21	23	22
11	B	28	29	31	30
10		24	25	27	26

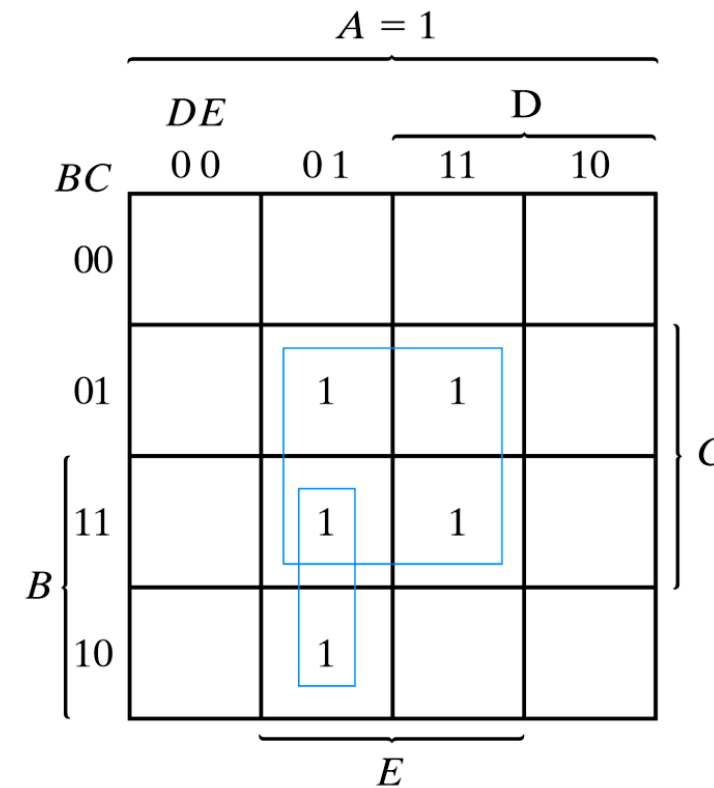
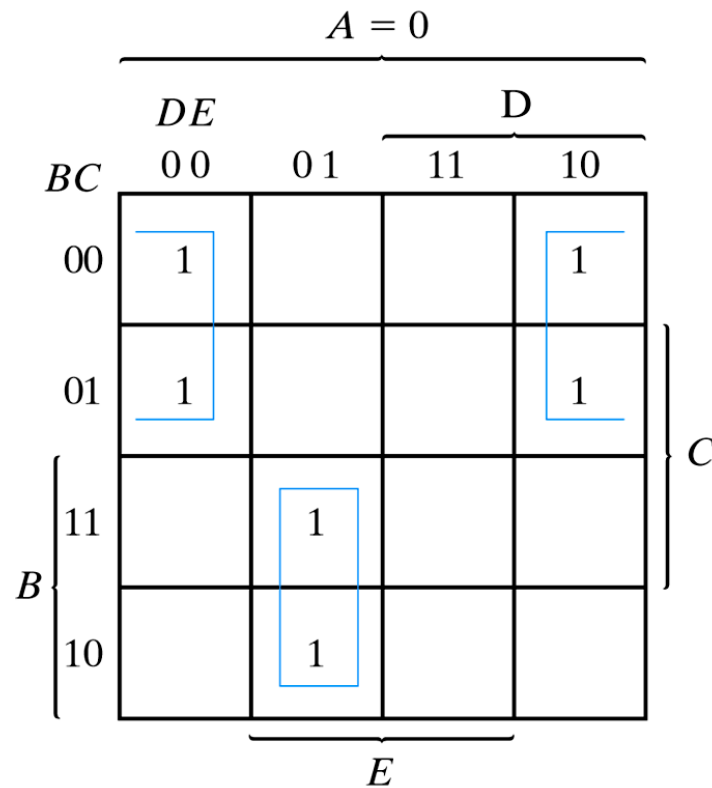
E

C

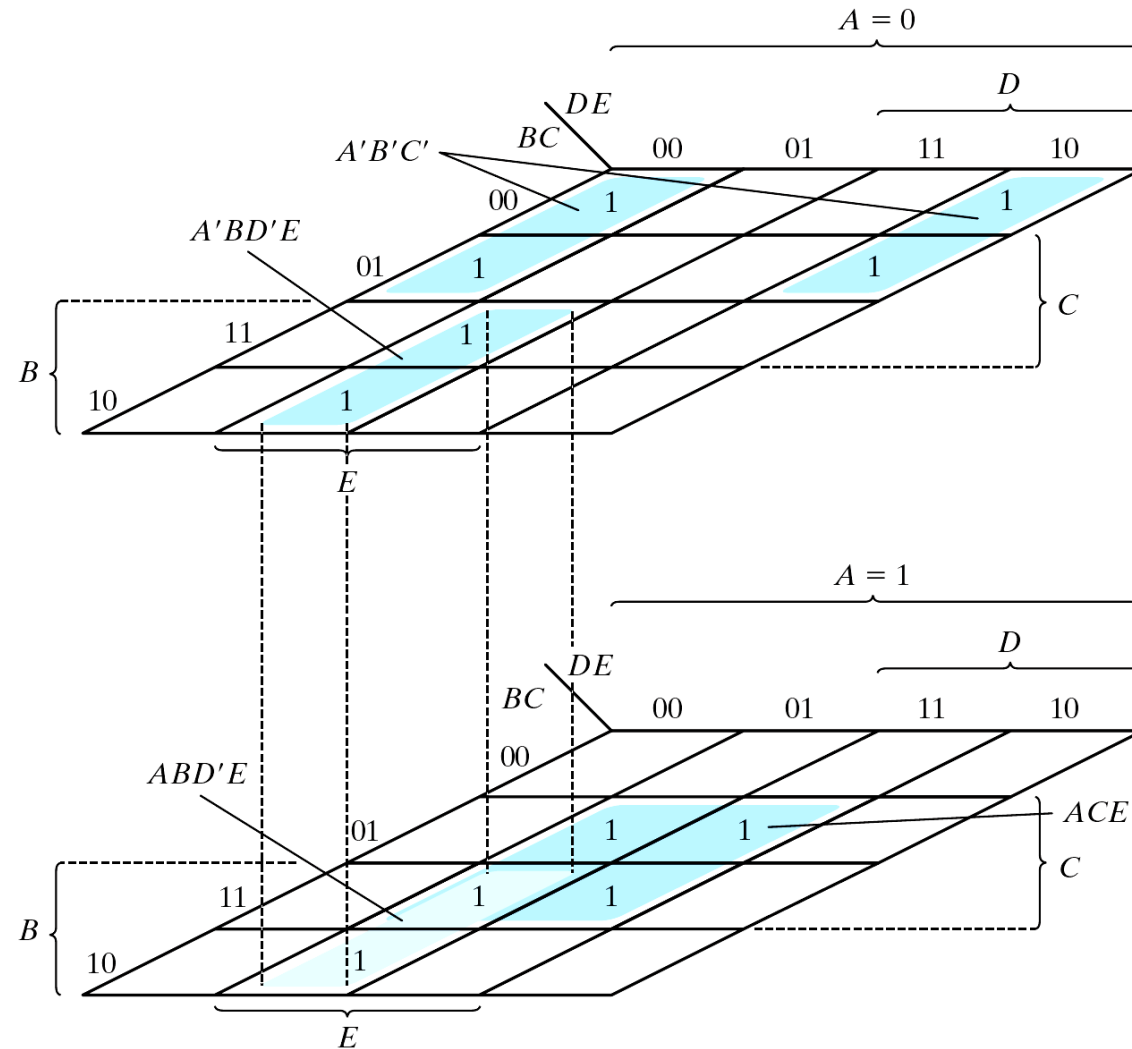
3.3.1 Five-Variable Map (2/3)

● Example 補充題

- $F(A, B, C, D, E) = \Sigma(0, 2, 4, 6, 9, 13, 21, 23, 25, 29, 31) = A'B'E' + BD'E + ACE$



3.3.1 Five-Variable Map (3/3)



3.4 Product of Sums Simplification (1/5)

- Approach #1
 - Simplified F' in the form of SOP
 - Apply DeMorgan's theorem $F = (F')'$
 - $F': \text{SOP} \rightarrow F: \text{POS}$
- Approach #2: duality
 - combinations of maxterms (it was minterms)
 - $M_0 M_1 = (A + B + C + D)(A + B + C + D') = (A + B + C) + (DD') = A + B + C$

3.4 Product of Sums Simplification (2/5)

- **Example 3.7** Simplify the Boolean function $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$ into (a) SOP form and (b) POS form

(a) Combine squares marked with 1's

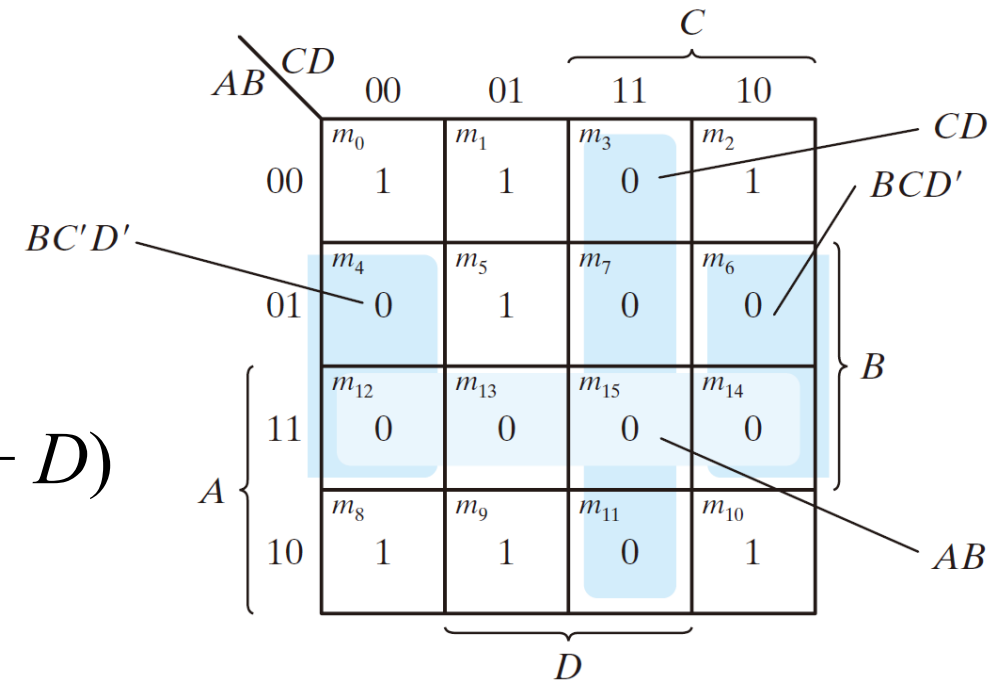
$$\text{SOP form} \rightarrow F = B'D' + B'C' + A'C'D$$

(b) Combine squares marked with 0's

$$\text{SOP form} \rightarrow F' = AB + CD + BD'$$

Then, apply the DeMorgan's theorem

$$\text{POS form} \rightarrow F = (A' + B')(C' + D')(B' + D)$$



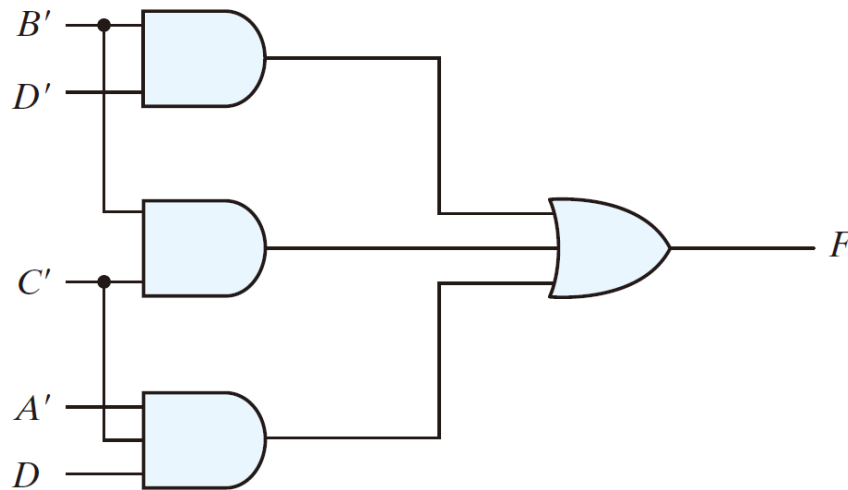
Note: $BC'D' + BCD' = BD'$

FIGURE 3.12

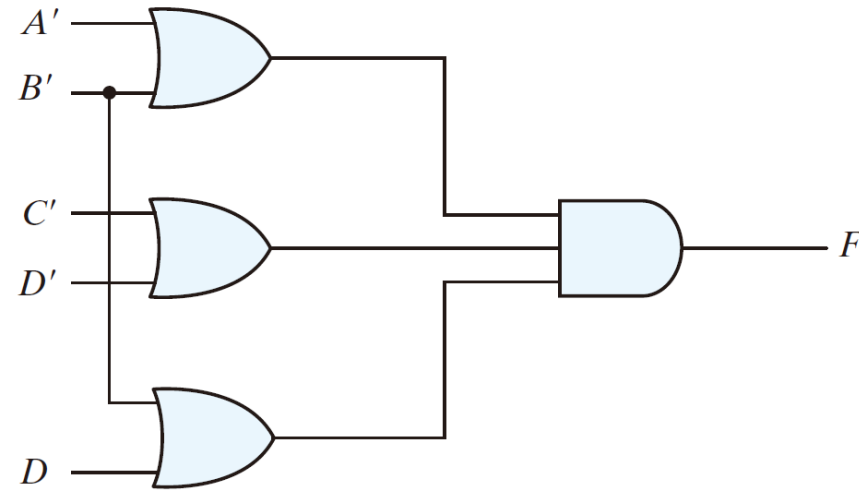
Map for Example 3.7, $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10) = B'D' + B'C' + A'C'D = (A' + B')(C' + D')(B' + D)$

3.4 Product of Sums Simplification (3/5)

- Gate implementation of the function of Example 3.7



(a) $F = B'D' + B'C' + A'C'D$



(b) $F = (A' + B')(C' + D')(B' + D)$

FIGURE 3.13

Gate implementations of the function of Example 3.7

3.4 Product of Sums Simplification (4/5)

- Consider the function defined in Table 3.1.
 - In sum-of-minterms: $F(x, y, z) = \Sigma(1, 3, 4, 6)$
 - In product-of-maxterms: $F(x, y, z) = \Pi(0, 2, 5, 7)$
 - Combining the 1's to obtain the SOP of F
 $\rightarrow F(x, y, z) = x'z + xz'$
 - Combining the 0's to obtain the SOP of F'
 $\rightarrow F'(x, y, z) = xz + x'z'$
- Taking the complement to obtain the POS
 $\rightarrow F(x, y, z) = (x' + z')(x + z)$

Table 3.1
Truth Table of Function F

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

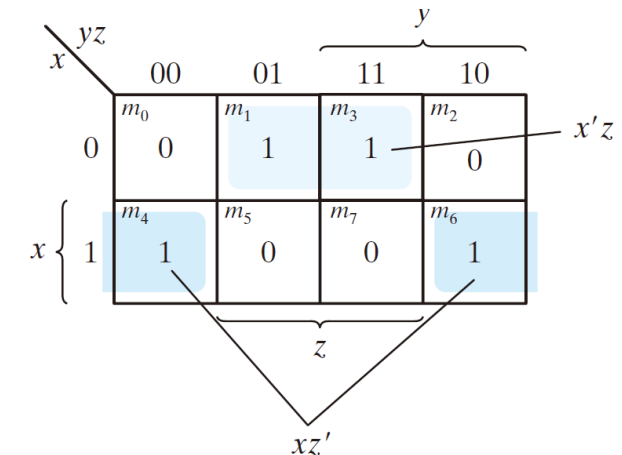


FIGURE 3.14
Map for the function of Table 3.1

3.4 Product of Sums Simplification (5/5)

- **Practice Exercise 3.8** Simplify the Boolean function $F(w, x, y, z) = \Sigma(0, 2, 8, 10, 12, 13, 14)$ into (a) SOP form and (b) POS form. Derive the truth table.

Answer:

- (a) $F(w, x, y, z) = x'z' + wz + wxy'$
- (b) $F'(w, x, y, z) = w'x + yz + x'z$

$$F(w, x, y, z) = (w + x')(y' + z')(x + z')$$

$wxyz$	F	$wxyz$	F
0000	1	1000	1
0001	0	1001	0
0010	1	1010	1
0011	0	1011	0
0100	0	1100	1
0101	0	1101	1
0110	0	1110	1
0111	0	1111	0

3.5 Don't-Care Conditions (1/3)

- The value of a function is not specified for certain combinations of variables
 - In BCD, 1010-1111 are regarded as don't care minterms.
- The don't care conditions can be utilized in logic minimization
 - can be implemented as 0 or 1
- In the K-map, an X is used in a square to represent a don't-care condition.

3.5 Don't-Care Conditions (2/3)

- **Example 3.8** Simply the Boolean function $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$ which has the don't care conditions $d(w, x, y, z) = \Sigma(0, 2, 5)$

Answer:

- $F = \Sigma(0, 1, 2, 3, 7, 11, 15)$
 - Figure 3.15(a): $F = yz + w'x'$
- $F = \Sigma(1, 3, 5, 7, 11, 15)$
 - Figure 3.15(b): $F = yz + w'z$
- either expression is acceptable
- Also apply to product of sums
 - $F' = z' + wy'$
 - $F = z(w' + y) = \Sigma(1, 3, 5, 7, 11, 15)$

		y			
		00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
11	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

(a) $F = yz + w'x'$

		y			
		00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
11	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0

(b) $F = yz + w'z$

FIGURE 3.15

Example with don't-care conditions

3.5 Don't-Care Conditions (3/3)

- **Practice Exercise 3.9** $F(w, x, y, z) = \Sigma(4, 5, 6, 7, 12)$ with don't care function $d(w, x, y, z) = \Sigma(0, 8, 13)$.
Answer: $F(w, x, y, z) = xy' + w'x = y'z' + w'x$

3.6 NAND and NOR Implementation (1/12)

- NAND gate is a universal gate
 - can implement any digital system

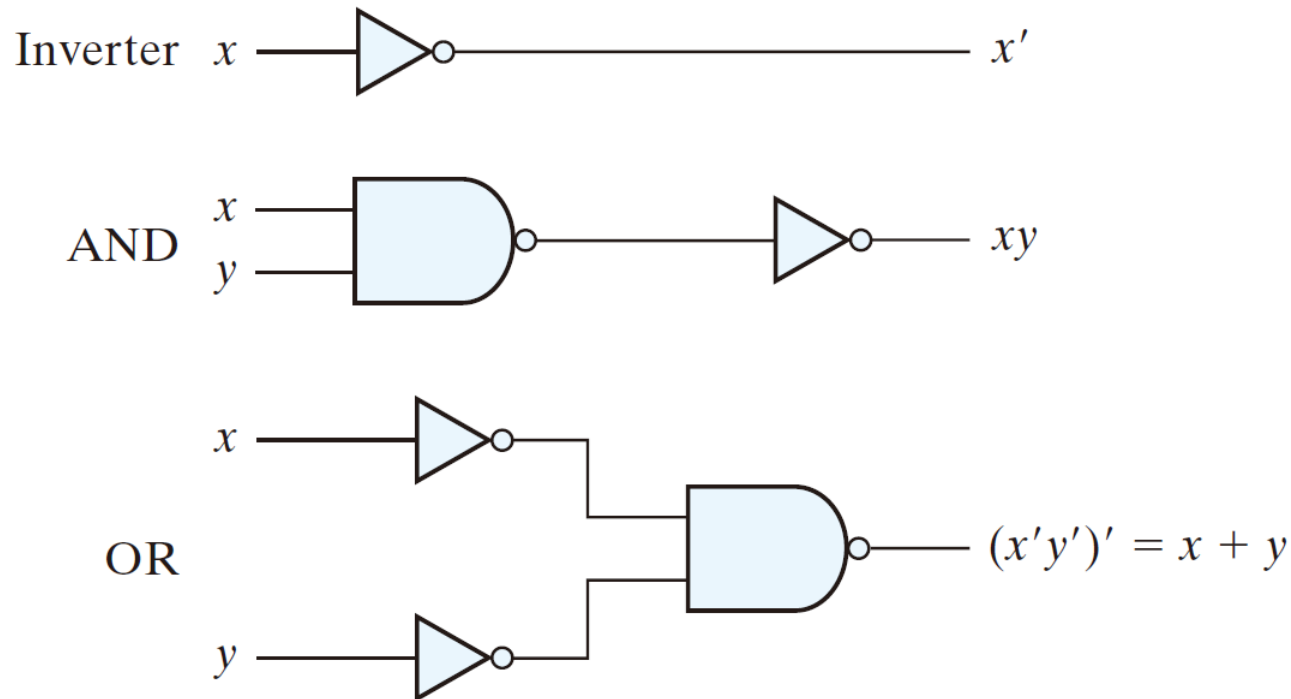


FIGURE 3.16
Logic operations with NAND gates

3.6 NAND and NOR Implementation (2/12)

- Two graphic symbols for a NAND gate

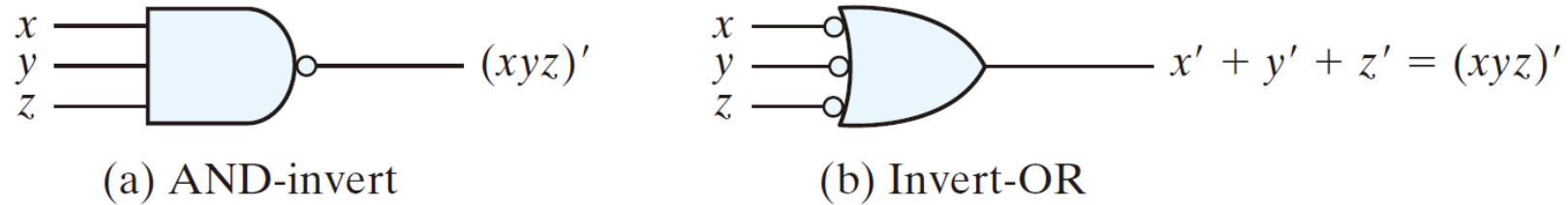


FIGURE 3.17

Two graphic symbols for a three-input NAND gate

3.6 NAND and NOR Implementation (3/12)

- Two-level Implementation
 - two-level logic
 - NAND-NAND = sum of products
 - Example: $F = AB + CD$
 - $F = ((AB)'(CD)')' = AB + CD$

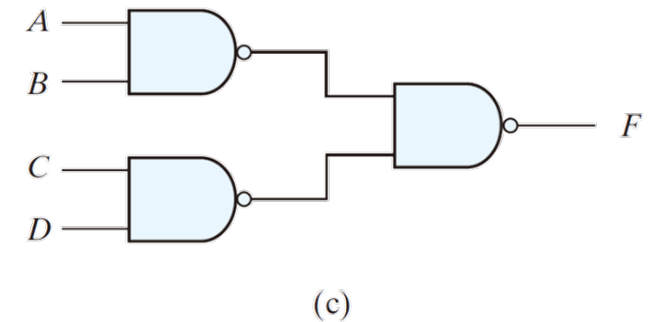
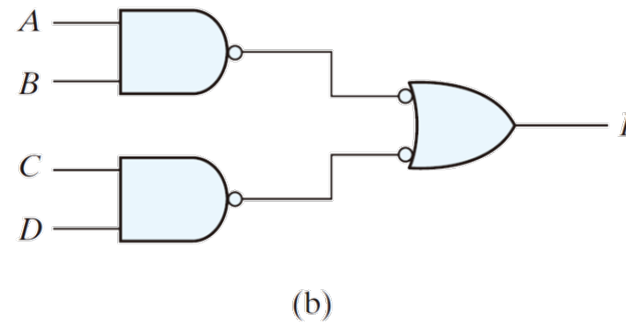
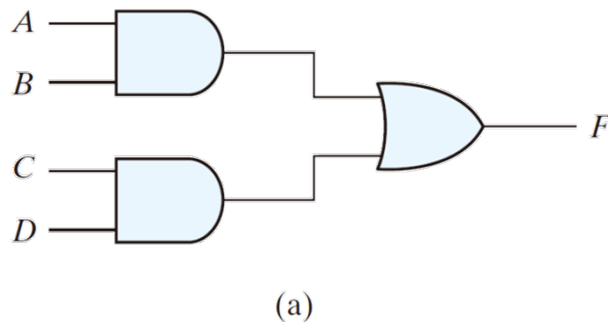


FIGURE 3.18
Three ways to implement $F = AB + CD$

3.6 NAND and NOR Implementation (4/12)

- **Example 3.9** Implement the Boolean function $F(x, y, z) = \Sigma(1, 2, 3, 4, 5, 7)$ with NAND gate.

Answer:

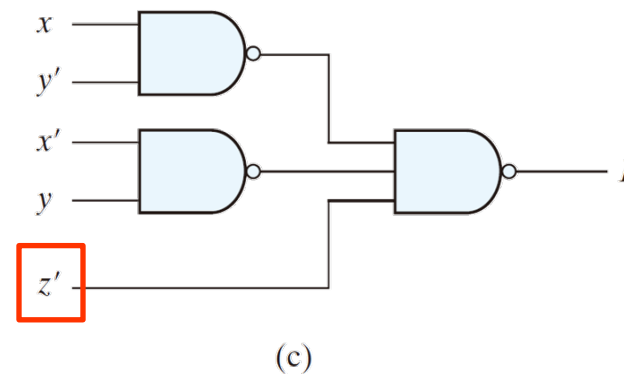
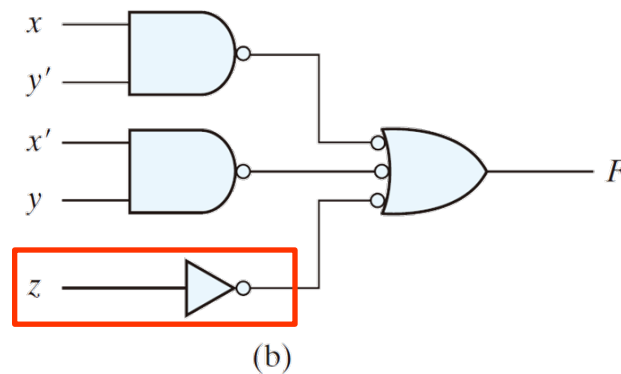
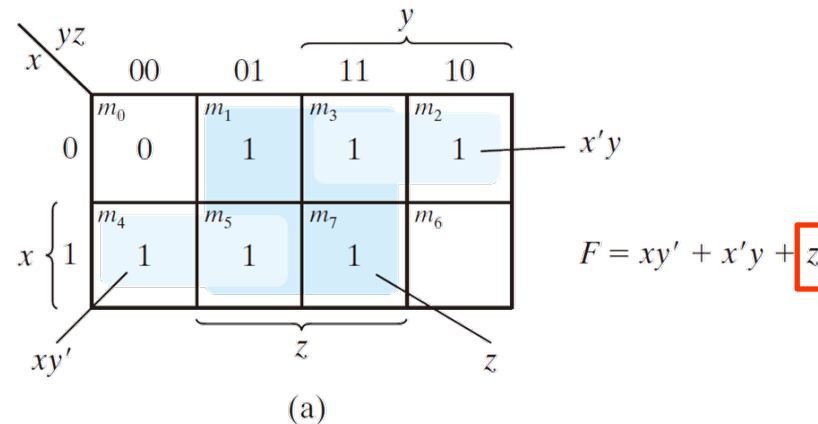


FIGURE 3.19
Solution to Example 3.9

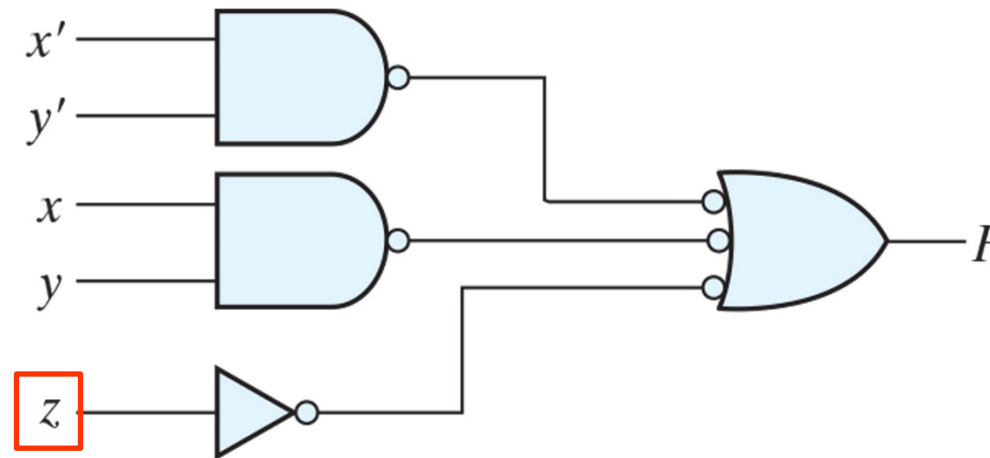
3.6 NAND and NOR Implementation (5/12)

- The procedure
 - simplified in the form of sum of products
 - a NAND gate for each product term; the inputs to each NAND gate are the literals of the term
 - a single NAND gate for the second sum term
 - **a single literal requires an inverter in the first level**

3.6 NAND and NOR Implementation (6/12)

- **Practice Exercise 3.10** Implement the Boolean function $F(x, y, z) = \Sigma(0, 1, 3, 5, 6, 7)$ with NAND gate and draw the logic diagram of the implementation.

Answer: $F(x, y, z) = x'y' + xy + \boxed{z}$



3.6 NAND and NOR Implementation (7/12)

- Multilevel NAND Circuits
 - Boolean function implementation
 - AND-OR logic $\rightarrow$ NAND-NAND logic
 - AND $\rightarrow$ AND + inverter $\rightarrow$ NAND
 - OR $\rightarrow$ inverter + OR $\rightarrow$ NAND
 - Compensate unpaired bubble by inserting an inverter or complementing the input literal

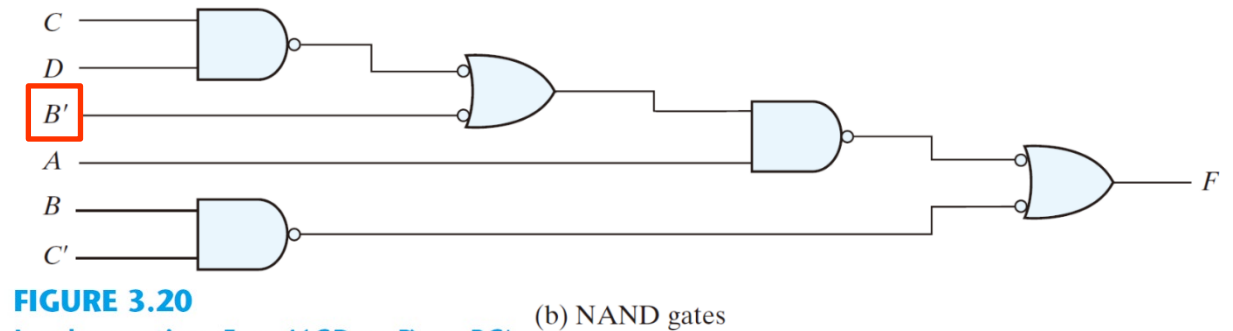
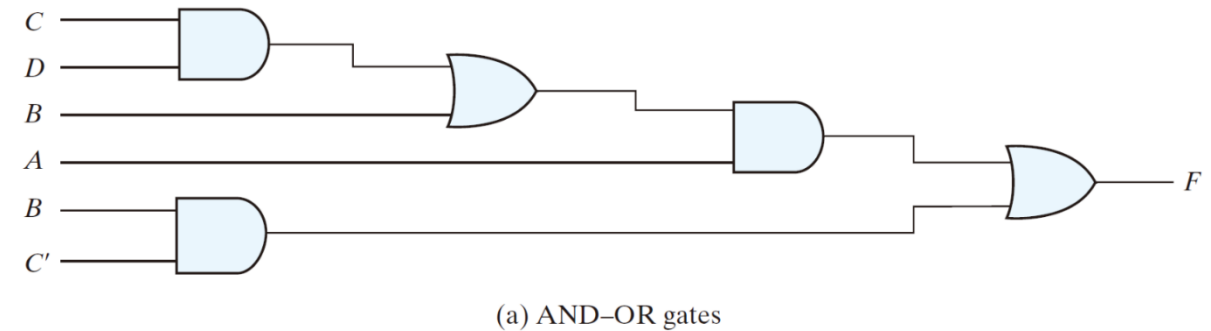


FIGURE 3.20
Implementing $F = A(CD + B) + BC'$

3.6 NAND and NOR Implementation (8/12)

- NAND implementation of $F = (AB' + A'B)(C + D')$

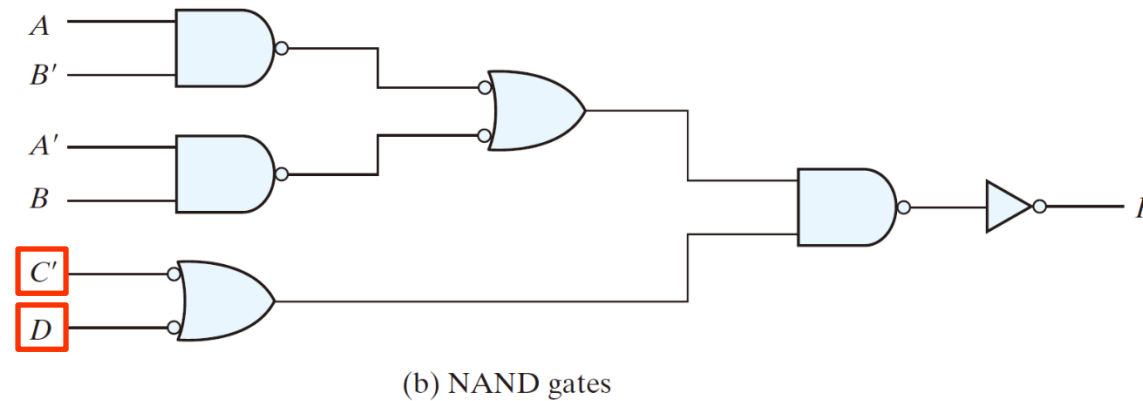
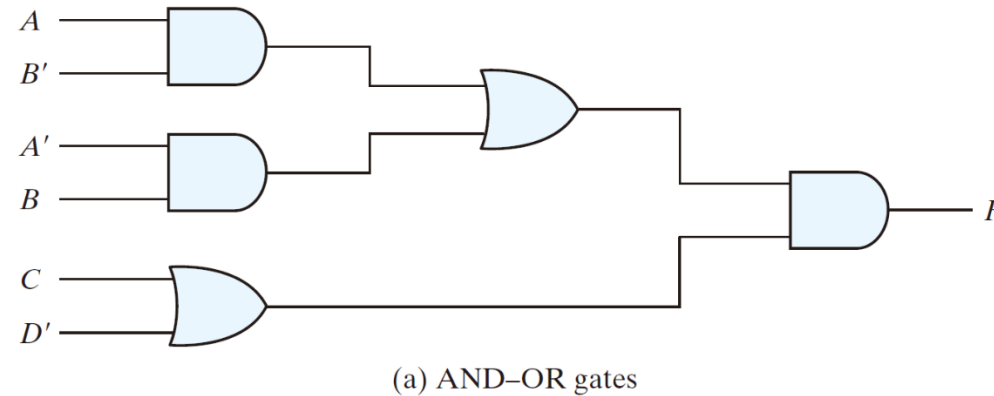


FIGURE 3.21

Implementing $F = (AB' + A'B)(C + D')$

3.6 NAND and NOR Implementation (9/12)

- NOR Implementation
- NOR function is the dual of the NAND function
- The NOR gate is also universal
- AND-OR logic $\rightarrow$ NOR-NOR logic
 - AND $\rightarrow$ inverter + AND $\rightarrow$ NOR
 - OR $\rightarrow$ OR + inverter $\rightarrow$ NOR
 - Compensate unpaired bubble by inserting an inverter or complementing the input literal

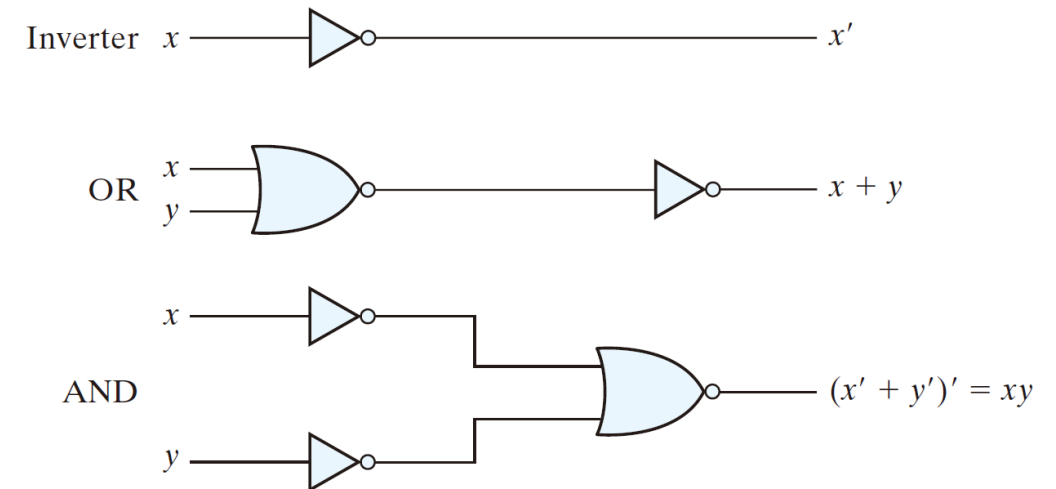


FIGURE 3.22
Logic operations with NOR gates

3.6 NAND and NOR Implementation (10/12)

- Two graphic symbols for a NOR gate

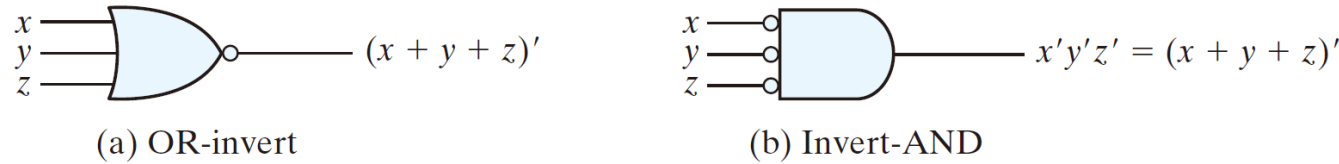


FIGURE 3.23

Two graphic symbols for the NOR gate

- Example: $F = (A + B)(C + D)E$

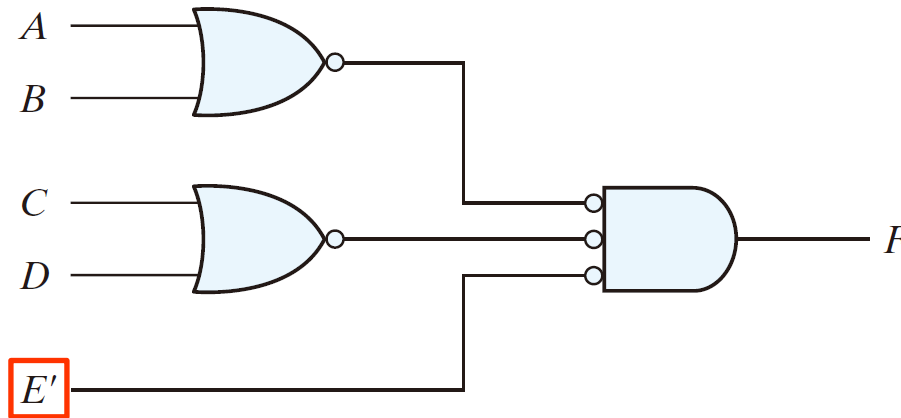


FIGURE 3.24

Implementing $F = (A + B)(C + D)E$

3.6 NAND and NOR Implementation (11/12)

- Example: NOR implement of $F = (AB' + A'B)(C + D')$

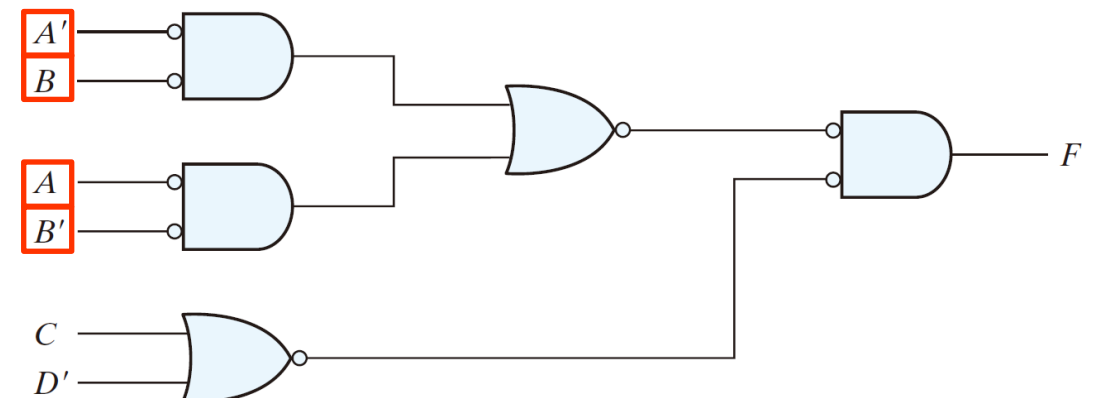
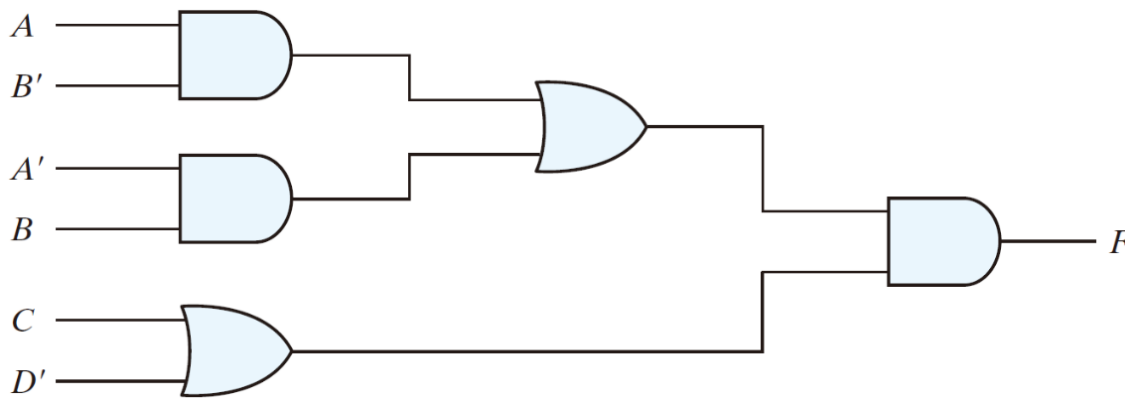


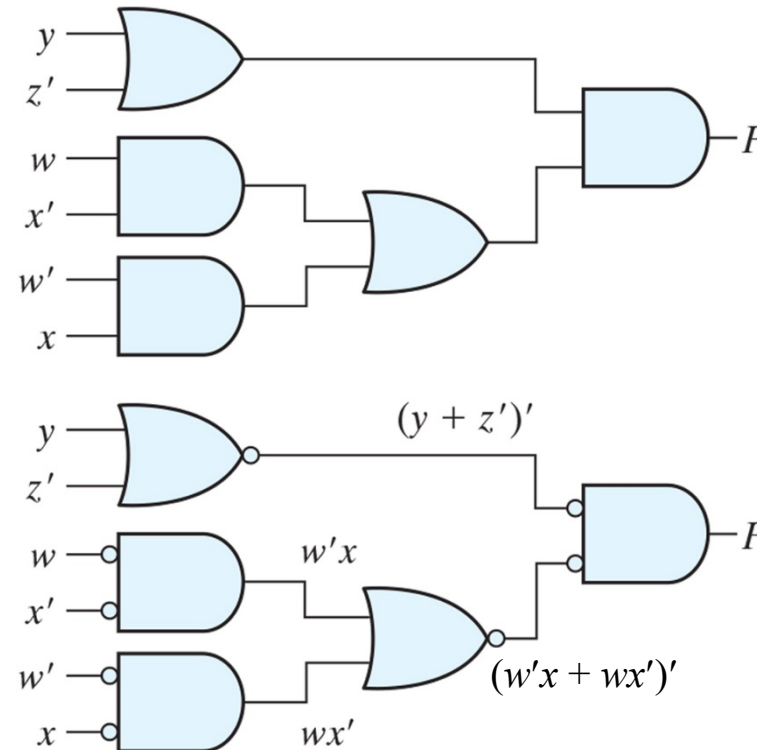
FIGURE 3.25

Implementing $F = (AB' + A'B)(C + D')$ with NOR gates

3.6 NAND and NOR Implementation (12/12)

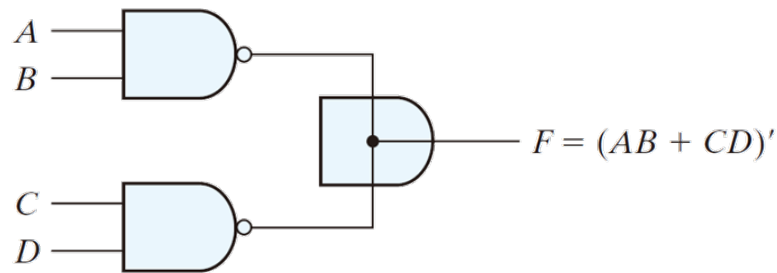
- **Practice Exercise 3.11** Implement the Boolean function $F(w, x, y, z) = (y + z')(wz' + w'x)$ with NOR gates.

Answer:

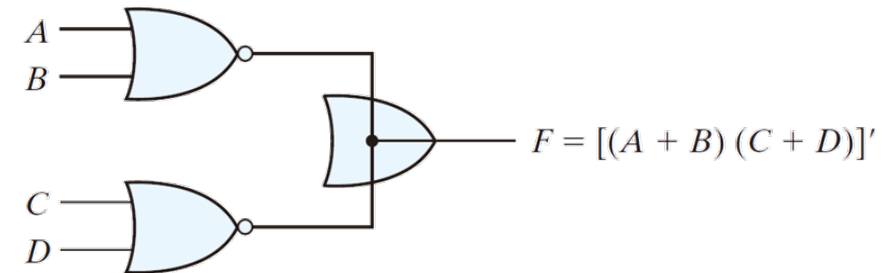


3.7 Other Two-Level Implementations (1/7)

- Wired logic (線結邏輯、線接邏輯)
 - a wire connection between the outputs of two gates
 - open-collector TTL NAND gates: wired-AND logic $\rightarrow$ AND-OR-INVERT
 - the NOR output of ECL gates: wired-OR logic $\rightarrow$ OR-AND-INVERT



(a) Wired-AND in open-collector
TTL NAND gates.
(AND-OR-INVERT)



(b) Wired-OR in ECL gates
(OR-AND-INVERT)

FIGURE 3.26

Wired logic

(a) Wired-AND logic with two NAND gates

(b) Wired-OR in emitter-coupled logic (ECL) gates

3.7 Other Two-Level Implementations (2/7)

- For four types of gates, 16 possible combinations of two-level forms
 - eight degenerate forms (退化形式) → a single operation
 - AND-AND, AND-NAND, OR-OR, OR-NOR, NAND-OR, NAND-NOR, NOR-AND, NOR-NAND
 - AND: AND-AND, NAND-NOR
 - OR: OR-OR, NOR-NAND
 - NAND: AND-NAND, NAND-OR
 - NOR: OR-NOR, NOR-AND
 - eight nondegenerate forms (非退化形式)
 - AND-OR, OR-AND, NAND-NAND, NOR-NOR, NOR-OR, NAND-AND, OR-NAND, AND-NOR
 - AND-OR and NAND-NAND = sum of products
 - OR-AND and NOR-NOR = product of sums
 - NOR-OR, NAND-AND, OR-NAND, AND-NOR = ?

3.7 Other Two-Level Implementations (3/7)

- NAND-AND = AND-NOR = AOI
- AND-OR-INVERT (AOI) implementation of $F = (AB + CD + E)'$
 - By combining 0's in the K-map to obtain **SOP** of F' , $F' = AB + CD + E$
 - Inverting F' to obtain the AOI implementation of F

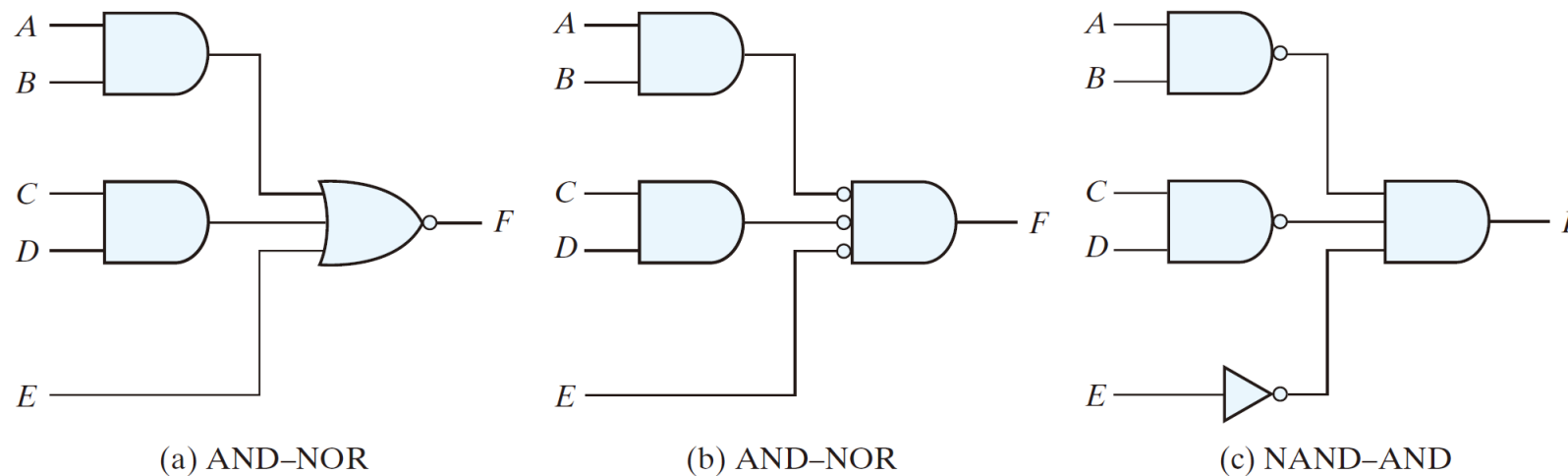


FIGURE 3.27
AND-OR-INVERT circuits, $F = (AB + CD + E)'$

3.7 Other Two-Level Implementations (4/7)

- OR-NAND = NOR-OR = OAI
- OR-AND-INVERT (OAI) implementation of $F = ((A + B)(C + D)E)'$
 - By combining 1's in the K-map to obtain **POS** of F' , $F' = (A + B)(C + D)E$
 - Inverting F' to obtain the OAI implementation of F

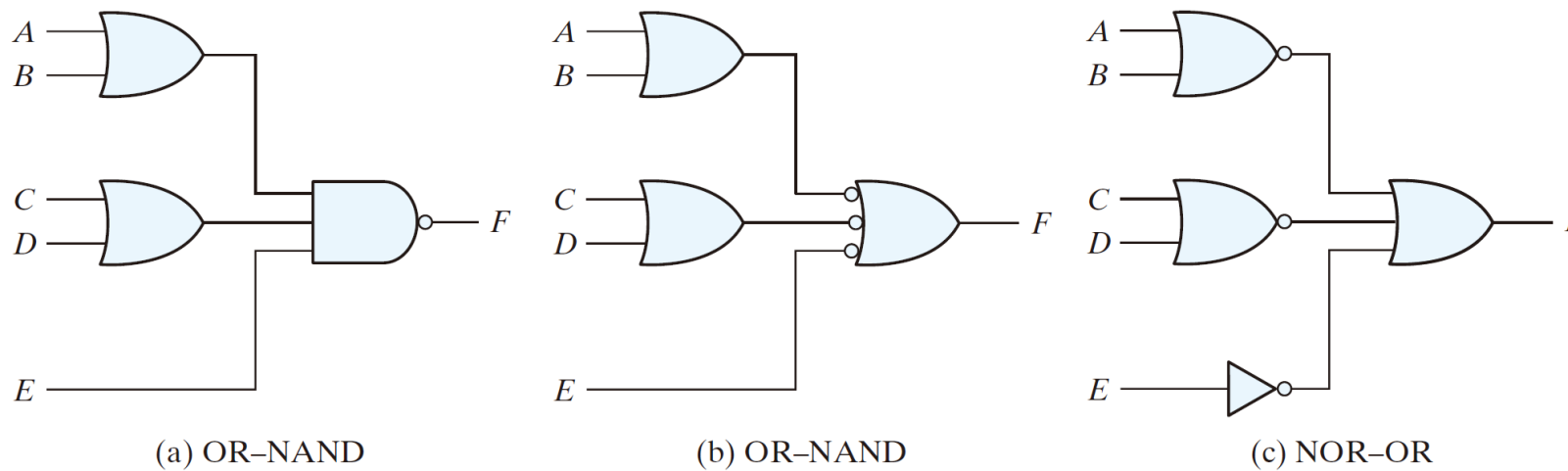


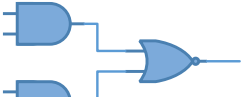
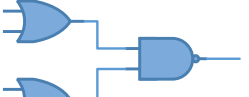
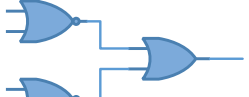
FIGURE 3.28

OR-AND-INVERT circuits, $F = [(A + B)(C + D)E]'$

3.7 Other Two-Level Implementations (5/7)

- Tabular Summary and Examples

Table 3.2
Implementation with Other Two-Level Forms

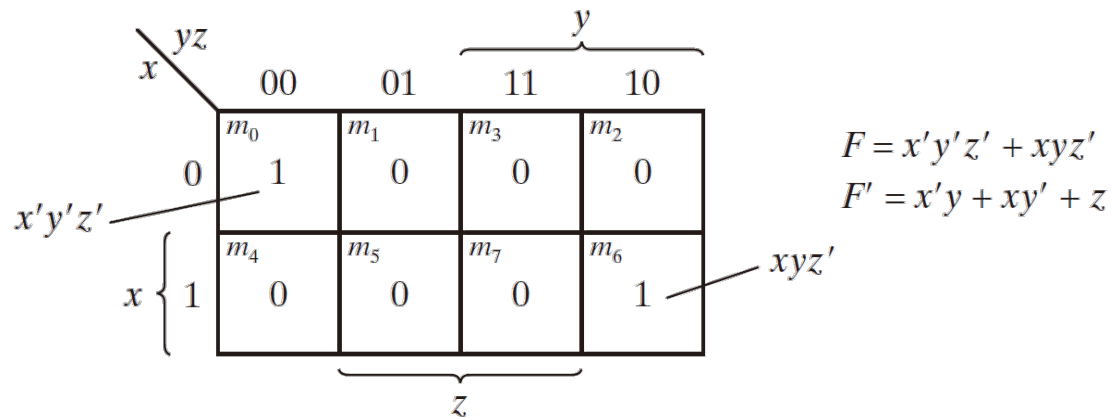
Equivalent Nondegenerate Form		Implements the Function	Simplify F' into	To Get an Output of
(a)	(b)*			
AND-NOR 	NAND-AND 	AND-OR-INVERT	Sum-of-products form by combining 0's in the map.	F
OR-NAND 	NOR-OR 	OR-AND-INVERT	Product-of-sums form by combining 1's in the map and then complementing.	F

*Form (b) requires an inverter for a single literal term.

3.7 Other Two-Level Implementations (6/7)

- **Example 3.10** Implement the function in Fig. 3.29(a) with the four 2-level forms listed in Table 3.2.

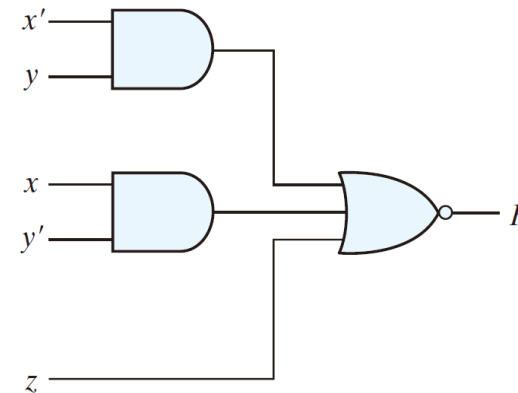
- $F' = x'y + xy' + z$ (F' : sum of products)
- $F = (x'y + xy' + z)'$ (F : AOI implementation)



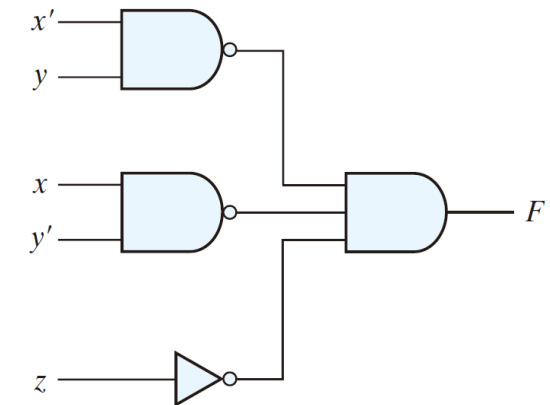
(a) Map simplification in sum of products

FIGURE 3.29

Other two-level implementations



AND-NOR

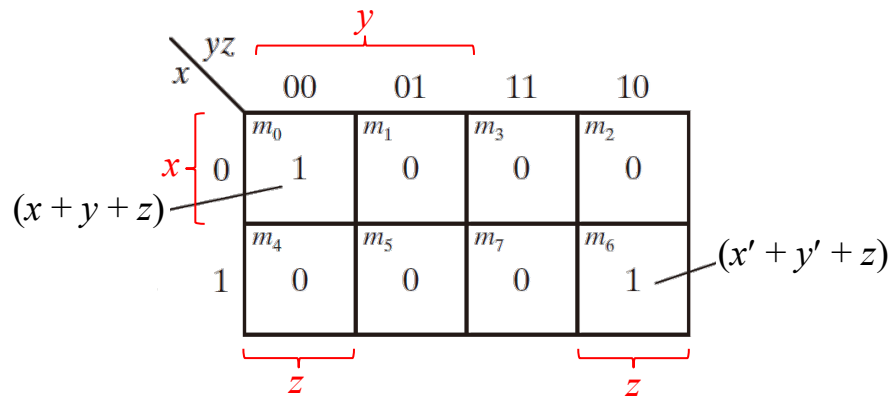


NAND-AND

(b) $F = (x'y + xy' + z)'$

3.7 Other Two-Level Implementations (7/7)

- $F' = (x + y + z)(x' + y' + z)$ (F' : product of sums)
- $F = ((x + y + z)(x' + y' + z))'$ (F : OAI)



Map simplification in product of sums

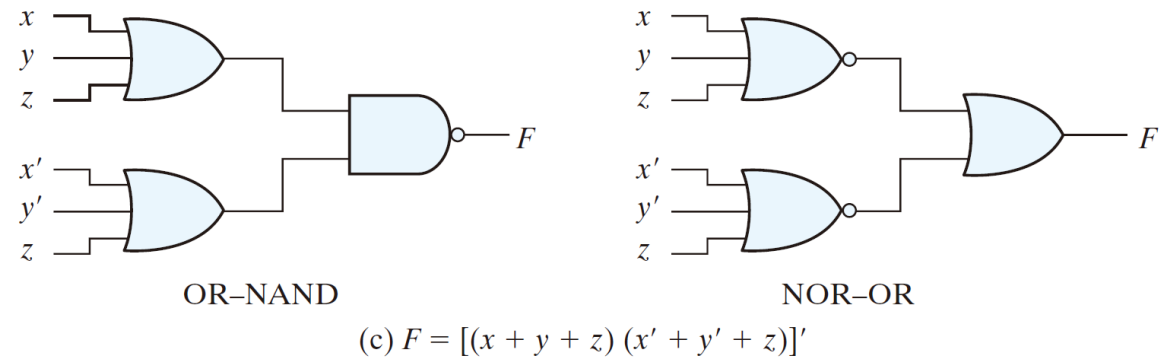


FIGURE 3.29
Other two-level implementations

3.8 Exclusive-OR Function (1/8)

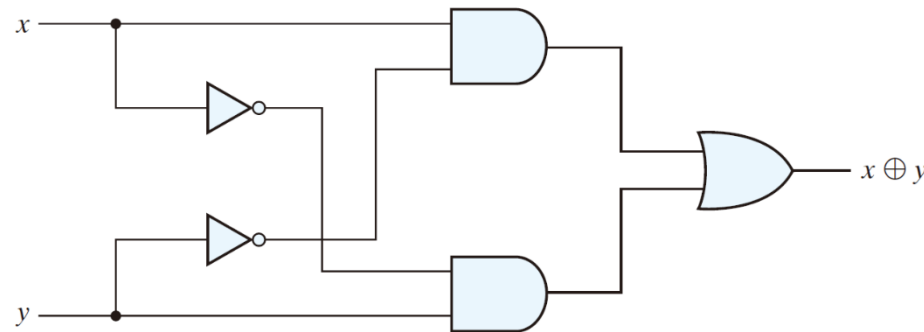
- Exclusive-OR (XOR)
 - $x \oplus y = xy' + x'y$
- Exclusive-NOR (XNOR)
 - $(x \oplus y)' = xy + x'y'$
- Commutative and associative
 - $A \oplus B = B \oplus A$
 - $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$
- Some identities
 - $x \oplus 0 = x$
 - $x \oplus 1 = x'$
 - $x \oplus x = 0$
 - $x \oplus x' = 1$
 - $x \oplus y' = (x \oplus y)'$
 - $x' \oplus y = (x \oplus y)'$

3.8 Exclusive-OR Function (2/8)

• Implementations

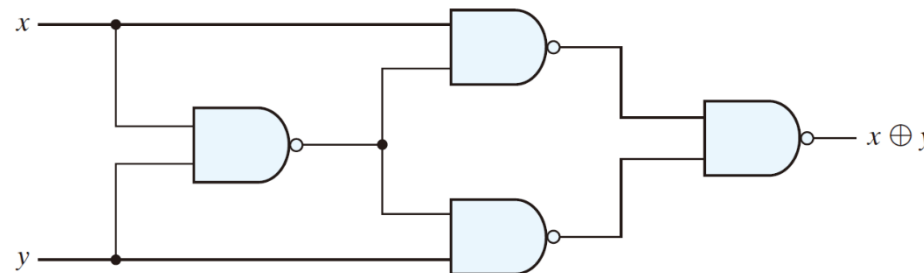
- $$(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$$

$$xy' + x'y$$



(a) Exclusive-OR with AND-OR-NOT gates

$$(x' + y')x + (x' + y')y$$



(b) Exclusive-OR with NAND gates

FIGURE 3.30
Exclusive-OR implementations

3.8 Exclusive-OR Function (3/8)

- Odd function

- $$A \oplus B \oplus C = (AB' + A'B)C' + (AB + A'B')C = AB'C' + A'BC' + ABC + A'B'C$$

$$= \Sigma(1, 2, 4, 7)$$
- an odd number of 1's

BC		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

(a) Odd function $F = A \oplus B \oplus C$

BC		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

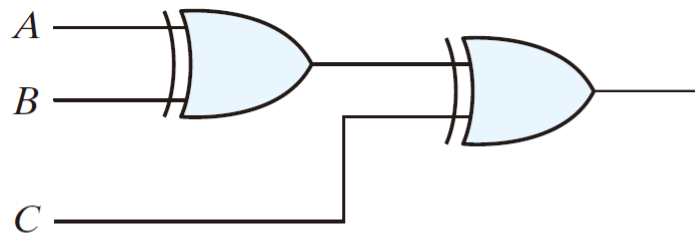
(b) Even function $F = (A \oplus B \oplus C)'$

FIGURE 3.31

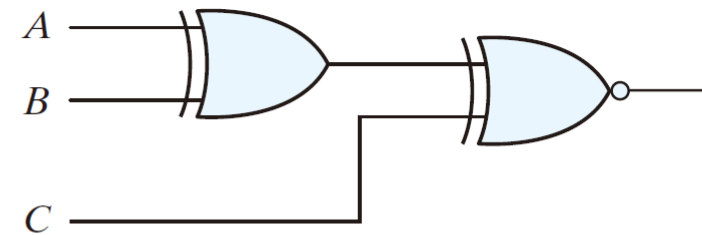
Map for a three-variable exclusive-OR function

3.8 Exclusive-OR Function (4/8)

- Logic diagram of odd and even functions



(a) 3-input odd function



(b) 3-input even function

FIGURE 3.32

Logic diagram of odd and even functions

3.8 Exclusive-OR Function (5/8)

- Four-variable Exclusive-OR function

- $$A \oplus B \oplus C \oplus D = (AB' + A'B) \oplus (CD' + C'D)$$

$$= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D)$$

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3	m_2 1
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12}	m_{13} 1	m_{15}	m_{14} 1
	10	m_8 1	m_9	m_{11} 1	m_{10}
		D			

(a) Odd function $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4	m_5 1	m_7	m_6 1
	11	m_{12} 1	m_{13}	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11}	m_{10} 1
		D			

(b) Even function $F = (A \oplus B \oplus C \oplus D)'$

FIGURE 3.33

Map for a four-variable exclusive-OR function

3.8 Exclusive-OR Function (6/8)

● Parity Generation and Checking

- generating parity bit: $P = x \oplus y \oplus z \rightarrow$ sending x, y, z, P
- checking parity check: $C = x \oplus y \oplus z \oplus P$
 - $C = 1 \rightarrow$ an odd number of data bit error
 - $C = 0 \rightarrow$ correct or an even # of data bit error

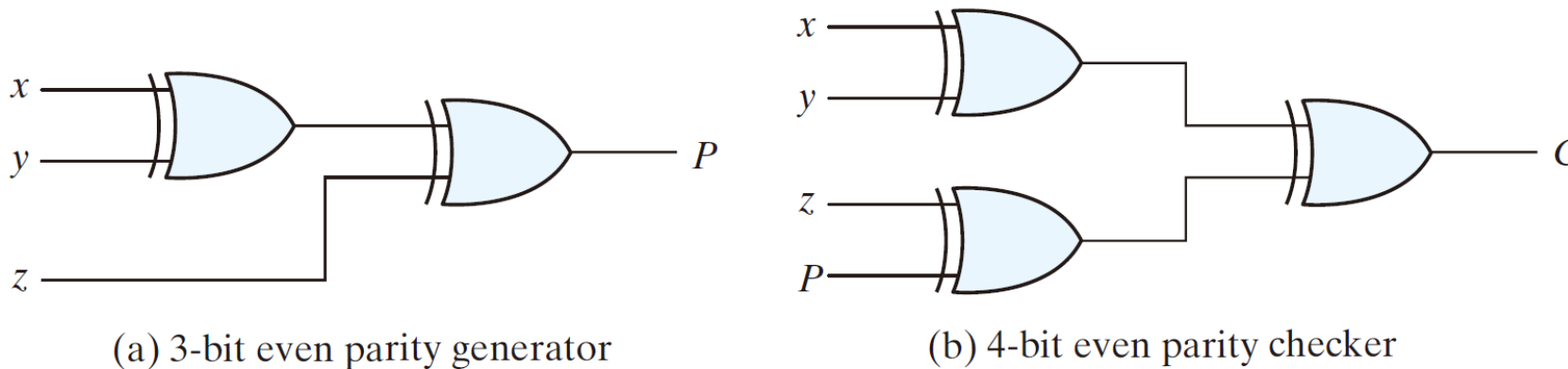


FIGURE 3.34

Logic diagram of a parity generator and checker

3.8 Exclusive-OR Function (7/8)

Table 3.3
Even-Parity-Generator Truth Table

Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

3.8 Exclusive-OR Function (8/8)

Table 3.4
Even-Parity-Checker Truth Table

Four Bits Received				Parity Error Check
<i>x</i>	<i>y</i>	<i>z</i>	<i>P</i>	<i>C</i>
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

3.9 Hardware Description Languages (HDLs) (1/2)

- A *hardware description language* (HDL) is a computer-based language that describes the hardware of digital systems in a textual form.
- As a *documentation language*, an HDL is used to represent and document digital systems in a form that can be read by both humans and computers and is suitable as an exchange language between designers.
- In VLSI circuit design, *fault simulation* compares the behavior of an ideal circuit with the behavior of a circuit that contains a process-induced flaw.
- In the public domain, IEEE supports standardized HDLs:
 - VHDL (**V**ery **H**igh Speed Integrated Circuit **HDL**): A US DoD-mandated language
 - Verilog
 - System Verilog: Evolved from Verilog and Super Log

3.9 Hardware Description Languages (HDLs) (2/2)

- HDLs are used in several major steps of the IC design flow:
 - *Design entry* (設計入門或設計輸入) creates an HDL-based description of the functionality that is to be implemented in hardware.
 - *Logic simulation* (邏輯模擬) displays the behavior of a digital system through the use of a computer.
 - The stimulus (i.e., the logic values of the inputs to a circuit) that tests the functionality of the design is called a *test bench* (測試平台).
 - *Logic synthesis* (邏輯合成) is the process of deriving a list of physical components and their interconnections, called a *netlist* (網表), from the model of a digital system described in an HDL.
 - *Timing verification* (時序驗證) confirms that the fabricated IC will operate at a specified speed.

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with Verilog (1/4)

- Design encapsulation (設計封裝), or design entry, creates a model representing the functionality of a digital circuit.
- A Verilog model is composed of text using about 100 keywords
 - **module**, **endmodule**, **input**, **output**, **wire**, **and (&&)**, **or (||)**, **not (!)**, ...
- A module, the text enclosed by the keyword pair **module ... endmodule**, is the fundamental descriptive (design) unit in the Verilog language.
- Format
 - Lines of text terminated with a semicolon (;).
 - Any text between two forward slashes (//) and the end of the line is a comment.
 - Multiline comments begin with /* and terminate with */.
 - Blank spaces are ignored.

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with Verilog (2/4)

Verilog Example 3.1

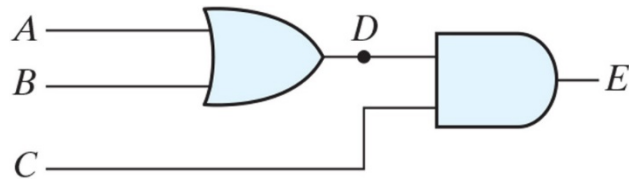


FIGURE 3.35 A logic diagram (schematic) for the Boolean equations $D = A + B$, $E = CD$.

Identifier

- module name, variables, ...
- composed of alphanumeric and underscore
- case sensitive
- may not start with a number

the mode (or direction) of a port is declared by the keyword **input**, **output**, or **inout**

```

module or_and (
    output E,
    input  A, B, C
);
    wire D;
    assign D = A || B;
    assign E = C && D;
    // This is a single-line comment
    /* The text here and below
       form a multi-line comment
    */
endmodule
  
```

module name is also the filename of the module, e.g., or_and

The port list is enclosed in parentheses.

The commas are used to separate elements of the list.

internal connection is declared by the keyword **wire**

assign D = A || B; // || is the logical "OR" operator

assign E = C && D; // && is the logical "AND" operator

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with Verilog (3/4)

- In Verilog, continuous assignment
 - It is used to represent the Boolean functions.
 - It doesn't prescribe a computation but the relationship between signals in a circuit.
 - It is specified by the keyword **assign**, followed by a Boolean expression.
- **Practice Exercise 3.12** Write a continuous assignment statement that describes Y in the logic diagram in Fig. PE3.12, where A , B , C , and D are Boolean variables.

Answer: `assign Y=((!A)||B)&&C`

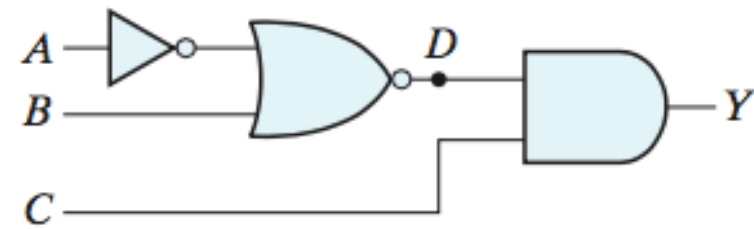


FIGURE PE3.12

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with Verilog (4/4)

- **Verilog Example 3.2** Develops a Verilog model of a circuit having inputs A, B, C, D and outputs E, F with functionality specified by the following Boolean expressions: $E = A + BC + B'D$ $F = B'C + BC'D'$

Answer:

```

module Circuit_Boolean_CA (E, F, A, B, C, D);
    output      E, F;
    input       A, B, C, D;
    assign      E = A || (B && C) || ((!B) && D);
    assign      F = ((!B) && C) || (B && (!C) && (!D));
endmodule

```

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (1/6)

- **VHDL (Optional)**
 - Entity
 - Provides a name by which a design can be identified
 - Specifies the interface of the design with its environment
 - Architecture
 - Describes how the outputs of an entity are formed from its inputs

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (2/6)

VHDL Example 3.1 (Optional)

Figure 3.36 depicts *or_and_vhdl*, a VHDL entity-architecture pair for a simple logic circuit. The entity identifies and specifies the mode and type of all of the inputs and outputs of the circuit. In VHDL the keyword names of allowed port modes (directions) are **in**, **out**, **inout**, and **buffer**. An **inout** port is bidirectional—one whose value can be generated within the architecture of the module and externally as well. A **buffer** declares that the port is an output but is also read within the module, for example, in a signal assignment.

The entity *or_and_vhdl* provides an interface between the architecture and its external environment. In this example the interface is a **port** consisting of three named *input* signals (*A*, *B*, and *C*) and one *output* signal (*E*). In general, a **port** statement identifies the input and output signals of the circuit, their direction (**in**, **out**, **inout**, **buffer**), and their data type (e.g., `std_logic`). Signals may be listed in any order in a port. The architecture here includes an internal signal, *D*, having type `std_logic`. Signal *D* connects the output

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (3/6)

(Optional)

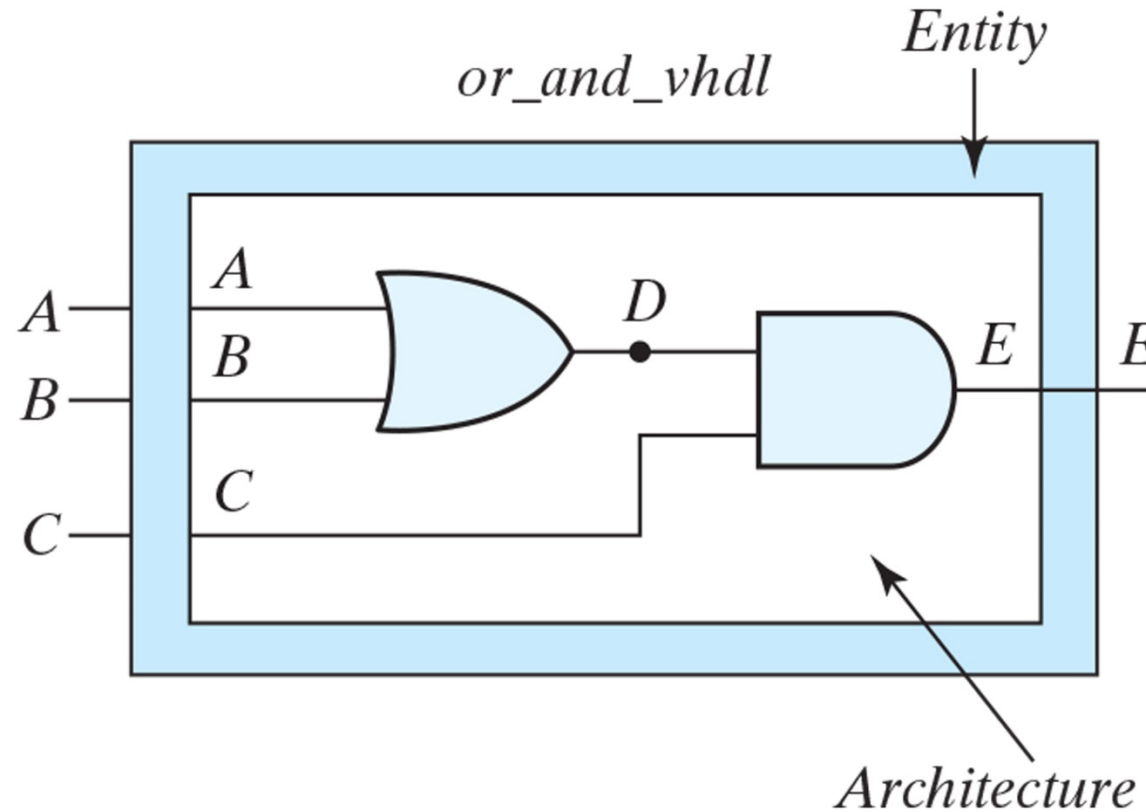


FIGURE 3.36

Entity-Architecture pair for *or_and_vhdl*.

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (4/6)

(Optional)

of the OR gate to an input of the AND gate, but is not part of the entity because it does not connect to the world outside *or_and_vhdl*. Signal *D* is not visible at the interface to the environment.

The descriptive style used in this architecture (below); is based on Boolean equations. It uses VHDL's built-in data operators⁹ to declare signal assignments using the Boolean expressions and equations implied by the schematic in Fig. 3.36. The signal assignment operator ($\leq$) and the accompanying Boolean expression specifies how a logic signal is formed from the values of other signals. The signal assignment statements in the architecture *Boolean_Equations* of entity *or_and_vhdl* specify how a simulator determines the values of *D* and *E* from *A*, *B*, and *C*.

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (5/6)

(Optional)

```
library ieee;
use ieee.std_logic_1164.all;

entity or_and_vhdl is
    port (A, B, C: in std_logic; E: out std_logic);
end or_and_vhdl;

architecture Boolean_Equations of or_and_vhdl is
    signal D: std_logic;
begin
    D <= A or B;
    E <= C and D;
end Boolean_Equations ;
```

3.9 Hardware Description Languages (HDLs)

Design Encapsulation and Modeling with VHDL (6/6)

(Optional)

- **Practice Exercise 3.13** Write a signal assignment statement that implements the logic diagram in Fig. PE3.13.

Answer: $F \leq (\text{not}((\text{not } A) \text{ or } B)) \text{ and } C;$

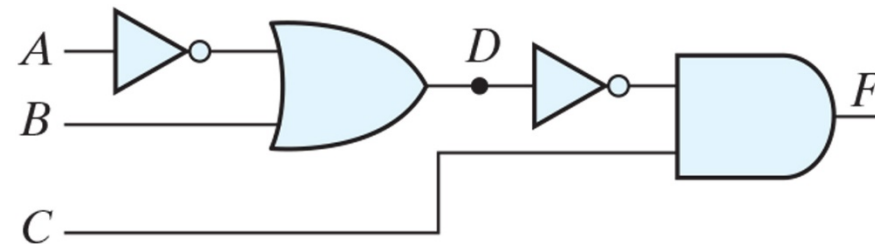


Figure PE 3.13

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- Verilog (1/4)

- Example 3.1 constructed Verilog and VHDL models based on the Boolean equations implied by the logic diagram of a circuit.
- Structural models describe how a circuit is composed of other interconnected elements, such as logic gates or functional blocks.
- Verilog has a family of built-in structural objects, called primitives, that enable direct modeling of combinational logic.
 - **and, nand, or, nor, xor, buf, not, bufif0, bufif1, notif0, notif1**

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- Verilog (2/4)

● Verilog Example 3.3 (Structural Modeling with Primitives)

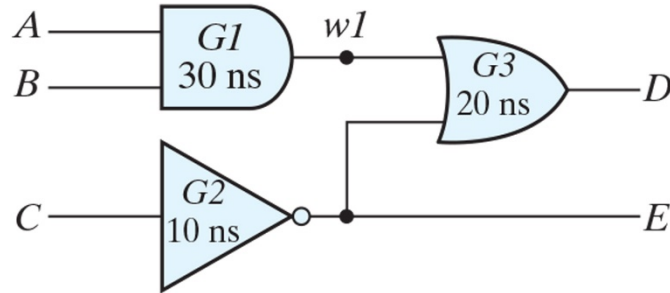


Figure 3.37
Schematic for *and_or_prop_delay*.

The association of time unit in Verilog with the physical time is made with the **'timescale** compiler directive.

→ **'timescale 1 ns / 1 ps** // time units / resolution

The keyword **wire** is used to declare internal connection between gates.

→ **module and_or_prop_delay (**
 input A, B, C;
 output D, E;
);

The propagation delay of a gate is specified in terms of time units and by the symbol #. The default time unit is 1 ns.

Gate instances or primitive instances. Each gate instance consists of a primitive name, an optional instance name followed by a list of comma-separated gate output and inputs enclosed within parentheses.

→ **wire w1;**

→ **and G1 #30 (w1, A, B);** // Prop delay: 30 ns
 not G2 #10 (E, C); // Prop delay: 10 ns
 or G3 #20 (D, w1, E); // Prop delay: 20 ns
endmodule

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- Verilog (3/4)

- An HDL description that provides the stimulus to a design is called a *test bench*.
- A test bench for *and_or_prop_delay* is given below

//Test bench for and_or_prop_delay

```
module t_and_or_prop_delay;           // The name of a test bench is generally with the prefix t_
wire   D, E; // The signals that are connected to the outputs of the circuit are declared with the keyword wire
reg    A, B, C; // The stimulus signals that are to be the inputs to the circuit are declared with the keyword reg
and_or_prop_delay M_UUT (A, B, C, D, E); // user-chosen instance name M_UUT (module unit under test) is required
initiate begin
    A = 1'b0; B = 1'b0; C = 1'b0;
    #100 A = 1'b1; B = 1'b1; C = 1'b1;
end
initial #200 $finish;
endmodule
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- Verilog (4/4)

- The order in which gates are referenced during simulation depends on the activity of the signals in the design, not on the order in which the statements are listed.

Table 3.5
Output of Gates after Delay

	Time Units (ns)	Input	Output		
		ABC	E	w1	D
Initial	—	0 0 0	1	0	1
Change	—	1 1 1	1	0	1
	10	1 1 1	0	0	1
	20	1 1 1	0	0	1
	30	1 1 1	0	1	0
	40	1 1 1	0	1	0
	50	1 1 1	0	1	1

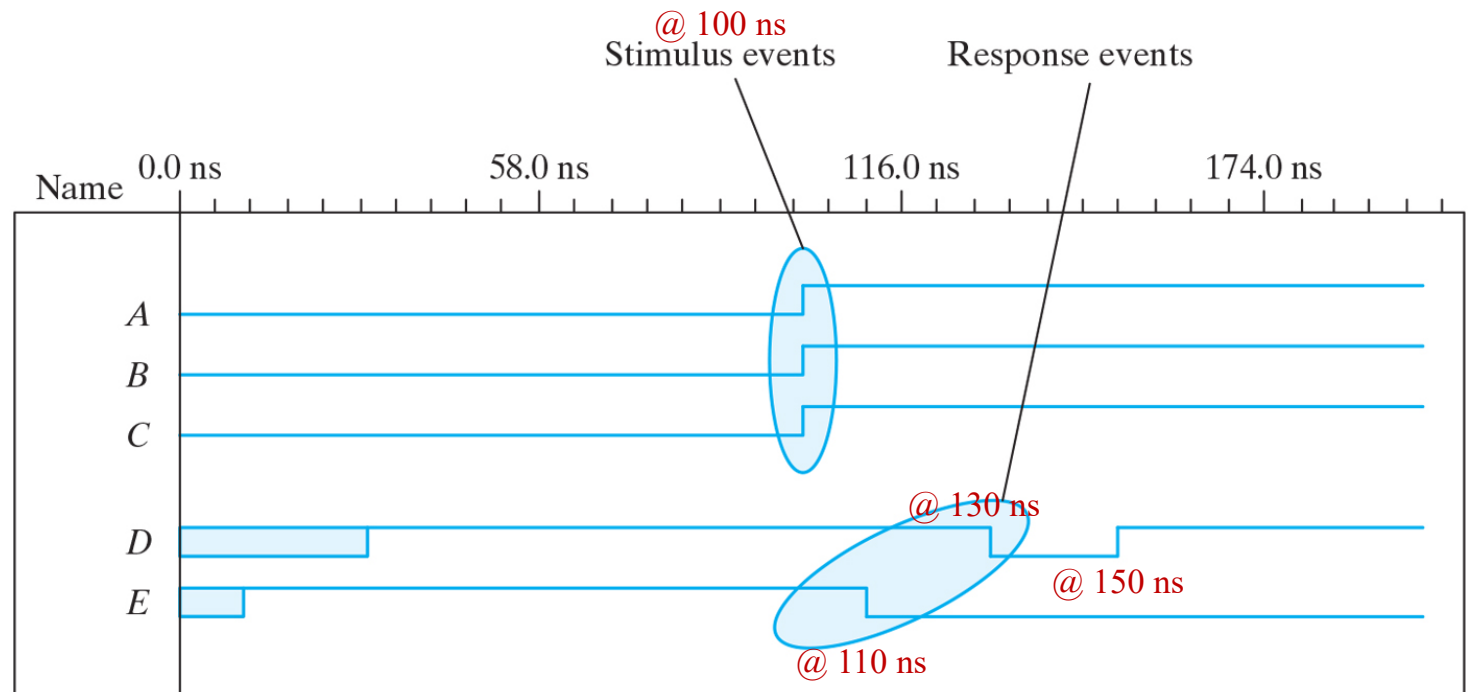


Figure 3.38
Simulation waveforms of *and_or_prop_delay*.

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (1/10)

(Optional)

VHDL Example 3.2 (Structural Modeling with Components)

VHDL does not have built-in combinational logic elements corresponding to logic gates; instead, the design units for structural modeling have to be built as “user-defined *components*,” which are modeled with the built-in language operators (the built-in binary logic operators are **and**, **or**, **nand**, **nor**, **xor**, and **xnor**). Once built, components can be used to build more complex structural models. Thus, structural modeling in VHDL is an indirect process of building components before building a structure using components.

The schematic shown in Fig. 3.39 is a structural description, that is, a connection of logic gates. To write a structural model of the circuit in VHDL code, we first build components *and2_gate*, *or2_gate*, and *inv_gate* observing the following syntax template and including the indicated propagation delays (optionally) in signal assignment statements.¹³

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (2/10)

(Optional)

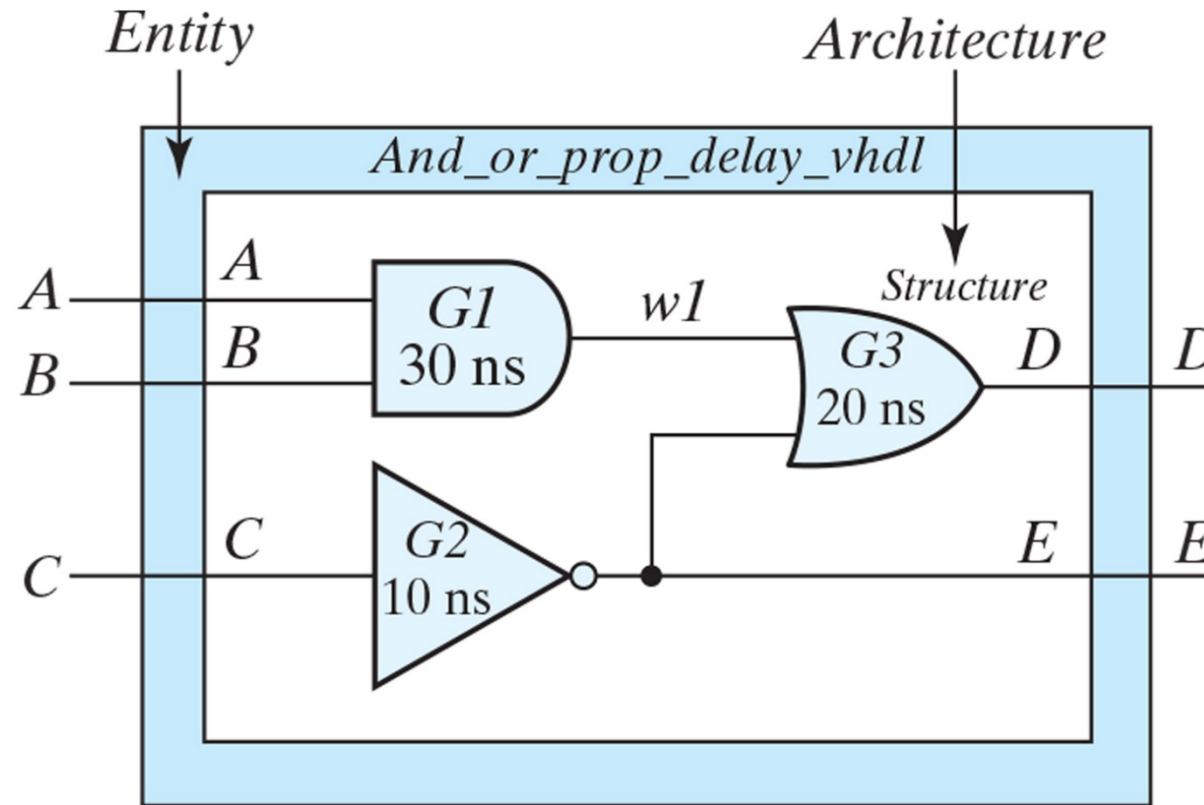


Figure 3.39

Entity-architecture for a structural model of *and_or_prop_delay_vhdl*.

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (3/10)

(Optional)

```

component component_name
    port (signal_names : mode signal_type;
        signal_names : mode signal_type;
        . . .
        signal_names : mode signal_type);
end component;

-- Model for 2-input and-gate component14

library ieee;
use ieee.std_logic_1164.all;

entity and2_gate is
    port (A, B: in std_logic; w1: out std_logic);
end and2_gate;
    
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (4/10)

(Optional)

```

end architecture Boolean_Operator;
-- Model for 2-input or-gate component

library ieee;
use ieee.std_logic_1164.all;

entity or2_gate is
    port (w1, E: in std_logic; D: out std_logic);
end or2_gate;

architecture Boolean_Operator of or2_gate is
begin
    D <= w1 or E after 20 ns; -- Logic operator
end architecture Boolean_Operator;

```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (5/10)

(Optional)

```
-- Model for inverter component

library ieee;
use ieee.std_logic_1164.all;

entity inv_gate is
    port (A: in std_logic; B: out std_logic);
end inv_gate;

architecture Boolean_Operator of inv_gate is
begin
    B <= not A after 10 ns;
end architecture Boolean_Operator;
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (6/10)

(Optional)

Next, we declare an entity-architecture pair for *and_or_prop_delay_vhdl*. The components to be used are listed with their ports; the internal signal *w1* is declared; and then components are instantiated to create a structural model.¹⁵ Each instantiation has a unique instance name (*G1*, *G2*, *G3*).

```
library ieee;
use ieee.std_logic_1164.all;

entity and_or_prop_delay_vhdl is
    port (A, B, C: in std_logic; D: out std_logic; E: buffer std_logic);
end Simple_Circuit_vhdl;

architecture Structure of and_or_prop_delay_is
    component and2_gate      -- Component declaration
        port (w1: out std_logic; A, B: in std_logic);
    end component;
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (7/10)

(Optional)

```
component or2_gate      -- Component declaration
  port (w1: out std_logic; A, B: in std_logic);
and component;
```

```
component inv_gate      -- Component declaration
  port (B: out std_logic; A: in std_logic);
end component;
```

```
signal w1: std_logic;
```

```
begin                                -- Component instantiations
  G1: and2_gate    port map (w1, A, B);
  G2: inv_gate     port map (E, C);
  G3: or2_gate     port map (D, w1, E);
```

```
end architecture Structure;
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (8/10)

(Optional)

Note: The port maps of the components in the preceding example associate by position the names of the ports in the instantiated component with the ports of the declared component. That mechanism is error-prone because it is easy to mistakenly write port names out of order. An alternative style, that is safer and essential when there are many signals in a port, associates the signals of a port's elements by name, in any order, that is, the *formal* names of the ports are associated with the *actual* names of the ports.¹⁶ For example, the gates of *Structure* can be instantiated as follows:

```
G1: and2_gate      port map (B => B, w1 => w1, A => A);
G2: inv_gate       port map (B => E, A => C);
G3: or2_gate       port map (A => w1, B => E, w1 => D);
```

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (9/10)

- VHDL Packages, Libraries, and Logic Systems **(Optional)**
 - *A package* provides a repository for declarations that may be common to several design units.
 - A package may have an optional body, which could contain declarations of components, as well as functions, and procedures supporting behavioral models.
 - *A package statement* has the syntax:

<pre> package package_name is [type_declarations] [signal_declarations] [constant_declarations] [component declarations] [function_declarations] [procedure_declarations] end package package_name; </pre>	<pre> package body package_name is [constant_declarations] [function_definitions] [procedure_definitions] end [package body][package_name]; </pre>
---	---

3.9 Hardware Description Languages (HDLs)

Structural (Gate-Level) Modeling --- VHDL (10/10)

(Optional)

Table 3.6

Logic Symbols of the IEEE_std_logic_1164 Package

'U'	Uninitialized
'X'	Strong drive, unknown logic value
'0'	Strong drive, logic 0
'1'	Strong drive, logic 1
'Z'	High impedance
'W'	Weak drive, unknown logic value
'L'	Weak drive, logic 0
'H'	Weak drive, logic 1
'_'	Don't care

3.10 Truth Tables In HDLs (1/3)

- Verilog—user-defined primitives
 - **and, or, etc., are** referred to as *system primitives*
 - *User-defined primitives* (UDPs) do not use the keyword pair **module...endmodule**; instead with the keyword pair **primitive...endprimitive**
- Verilog Example 3.4 defines a UDP with a truth table
 - A UDP is declared with the keyword **primitive**, followed by a name and port list
 - There can be only one output, and it must be listed in the port list and declared with the keyword **output**

3.10 Truth Tables In HDLs (2/3)

Verilog Example 3.4 (User-Defined Primitive)

```
// Verilog model: User-defined Primitive
primitive UDP_02467 (D, A, B, C);
    output D;
    input  A, B, C;
    //Truth table for D = f (A, B, C) =  $\sum(0, 2, 4, 6, 7)$ ;
    table
//A  B  C  :    D      // Column header comment
0  0  0  :    1;
0  0  1  :    0;
0  1  0  :    1;
0  1  1  :    0;
1  0  0  :    1;
1  0  1  :    0;
1  1  0  :    1;
1  1  1  :    1;
    endtable
endprimitive
// Instantiate primitive
// Verilog model: Circuit instantiation of Circuit_UDP_02467
module Circuit_with_UDP_02467 (e, f, a, b, c, d);
    output      e, f;
    input       a, b, c, d;
    UDP_02467   (e, a, b, c);
    and         (f, e, d); // Optional gate instance name omitted
endmodule
```

3.10 Truth Tables In HDLs (3/3)

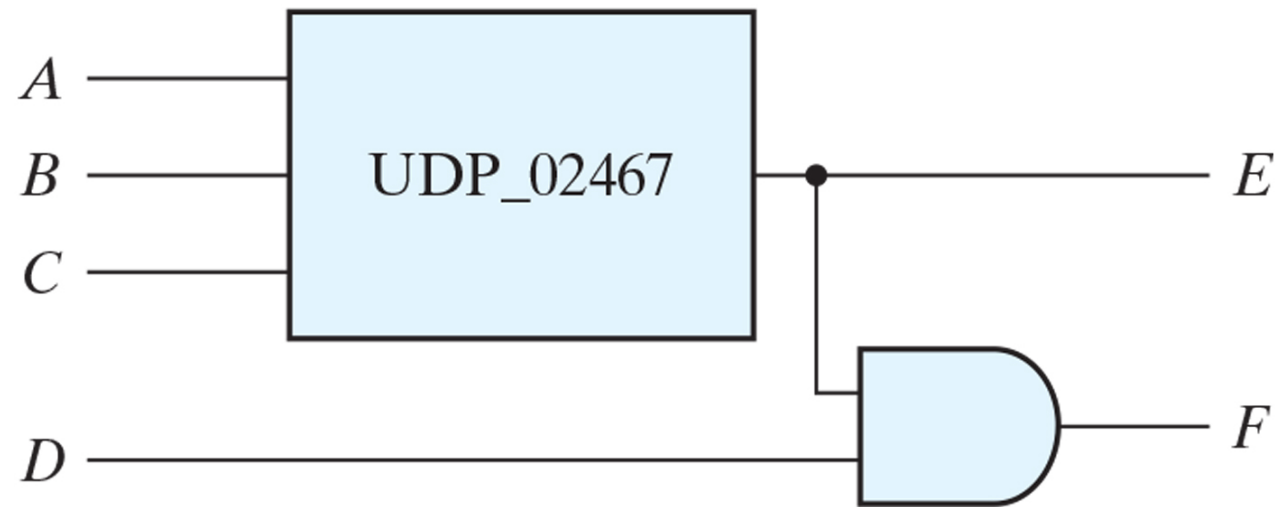


Figure 3.40
Schematic for *Circuit with _UDP_02467*.